台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第四篇

第 13 ～ 16 章

2D SLAM 與掃地機器人

李世淵 (機器人叫獸)

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第四篇 2D SLAM 與掃地機器人

第 13 章

2D 地圖建構

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第三篇把工具都備齊了：第 10 章的座標轉換、第 11 章的機率估測、第 12 章的最佳化。從本章開始，第四篇要把這些工具真正組裝成一套會動的系統。

而第一件事，是把感測器量到的東西變成一張地圖。本章的佔據格地圖是全書第一種「真的可以存進檔案、交給導航系統使用」的地圖格式——第 14、15 章的 SLAM 系統輸出它，第 16 章的掃地機器人靠它規劃清掃路徑，第 29 章的 Nav2 把它當成導航堆疊的標準輸入。

本章刻意做了一個簡化：假設位姿是已知的。這當然不成立——第 1 章就說過那是雞生蛋問題——但先把「已知位姿怎麼建圖」弄清楚，第 14、15 章才能專心處理「位姿不準怎麼辦」。這是教學上必要的拆解。

圖 13-0　第 13 章與全書的連結地圖

【承接前章】

第 10 章　座標轉換　→　把光達點從機體座標搬到世界座標

第 11 章　機率　　→　每一格存的不是 0 / 1，而是佔據機率

第 8 章　　光達規格　→　角解析度決定地圖能畫多細

【本章負責】

佔據格地圖的資料結構　→　對數勝算比更新　→　光線投射

掃描匹配（ICP）　→　里程計與光達的融合

解析度取捨、地圖品質評估

【往下延伸】

→ 第 14 章　Gmapping：把位姿也一起估（粒子濾波路線）

→ 第 15 章　Cartographer：子圖 + 分支定界閉環（最佳化路線）

→ 第 16 章　掃地機器人：佔據格地圖的消費級應用

→ 第 17 章　ICP 的三維版本：點雲配準

→ 第 29 章　Nav2 以本章的地圖為輸入做路徑規劃

學習目標

前三篇打好了觀念與數學地基，從本篇開始，理論要在真實機器人上落地了。第四篇聚焦最親民、也最貼近生活的 2D SLAM——掃地機器人用的正是這套技術。本章先講清楚「2D 地圖是什麼、怎麼建」，為後兩章的 Gmapping 與 Cartographer 鋪路。學完本章之後，同學應能夠達成下列目標：

1. 說明佔據格地圖的結構，以及每一格「佔據機率」的意義與更新方式。
2. 說明雷射掃描匹配如何估計機器人的相對移動。
3. 解釋為什麼要融合里程計與光達，兩者如何互補。

4. 說明地圖解析度的選擇如何影響地圖品質、記憶體與運算。
5. 分析走廊、教室、辦公室等典型室內場景的 2D 建圖挑戰。
6. 說明如何評估一張 2D 地圖的品質，辨識變形、斷裂與漂移等常見問題。

核心概念

中文名詞	英文名詞	概念說明
佔據格地圖	Occupancy Grid Map	把空間切成規則格子，每格記錄被佔據的機率。
佔據機率	Occupancy Probability	某格子被障礙物佔據的機率，介於 0 與 1 之間。
對數勝算比	Log-odds	以對數形式表示機率，使地圖更新變成簡單的加減法。
掃描匹配	Scan Matching	比對前後兩次雷射掃描，估計機器人的相對移動。
ICP	Iterative Closest Point	反覆找對應點並對齊的掃描匹配經典演算法。
里程計	Odometry	由輪速估計的相對位移，連續但會漂移。
光線投射	Ray Casting	沿雷射光路徑更新途經格子的方法。
地圖解析度	Map Resolution	每個格子代表的實際邊長，如 5 公分。
自由空間	Free Space	雷射穿過、確認無障礙的區域。
未知區域	Unknown	感測器尚未觀測到的區域。
地圖漂移	Map Drift	因累積誤差導致地圖逐漸偏移或變形。
逆感測器模型	Inverse Sensor Model	由一次量測推算沿途各格佔據機率的模型。

13.1 Occupancy Grid Map

第 1 章與第 3 章我們初識過佔據格地圖，本節深入它的運作機制——這是 2D SLAM 最核心的地圖表示法，掃地機器人 App 上那張房間地圖，底層就是它。

把空間切成棋盤

佔據格地圖（Occupancy Grid Map）的想法極其樸素：把機器人活動的平面切成一格一格的方格（像棋盤或方格紙），每一格記錄一個數字——這一格「被障礙物佔據的機率」。

- 機率接近 1（習慣畫成黑色）：幾乎確定有障礙，例如牆、桌腳。
- 機率接近 0（畫成白色）：幾乎確定是空的，可以通行。
- 機率為 0.5（畫成灰色）：不知道——感測器還沒看過這一格。

這三種狀態（佔據、自由、未知）的區分至關重要。特別是「未知」——它不等於「自由」。地圖邊緣那一大片灰色，代表機器人還沒探索的地方，不能當成可通行區域，否則規劃路徑時會「以為那裡是空的」而出錯。這是佔據格地圖比單純「有/無障礙」二元地圖高明的地方。

每一格獨立處理

佔據格地圖有一個關鍵假設：每一格的佔據狀態互相獨立。這在數學上並不完全成立（相鄰格子顯然相關——牆是連續的），但這個簡化讓計算變得極其高效：每來一筆雷射量測，只需更新它掃到的那幾格，不必考慮格子之間的關聯。這是工程上「用合理的簡化換取可行的計算」的又一個範例，與第 11 章佔據格地圖採高斯假設的精神一致。

前提：必須先知道位姿

這裡要強調一個第 1 章反覆出現的觀念：建佔據格地圖的前提，是已知機器人每一刻的位姿。因為一筆雷射量測「前方三公尺有牆」，只有知道機器人在哪、朝哪，才能決定這面牆該畫在地圖的哪一格。

於是問題來了——位姿從哪來？這正是 SLAM 的雞生蛋問題。本章先假設「位姿已由某種方式提供」（例如里程計加掃描匹配），專注於「有了位姿，如何建圖」；至於「如何同時估好位姿」，是第 14、15 章 Gmapping 與 Cartographer 的核心。把「建圖」與「定位」暫時拆開理解，是學習 SLAM 的有效策略。

13.2 佔據機率的更新：對數勝算比

機器人不會只看一眼就下結論——它會反覆掃描同一個地方，每次掃描都提供新證據，地圖應該把這些證據累積起來。本節說明這個累積是怎麼做的，以及一個讓計算變簡單的巧妙技巧。

累積證據：貝氏更新

每一格的佔據機率，隨著一次次觀測而更新。若某格連續多次被雷射打到（每次都說「這裡有東西」），它的佔據機率就逐漸逼近 1;若連續多次被雷射穿過（每次都說「這裡是空的」），機率就逼近 0。這正是第 11 章貝氏更新的應用——用新證據修正既有信念。

但直接對機率做貝氏更新，需要反覆做乘法與除法（還要正規化），計算較繁瑣，而且機率被限制在 0 到 1 之間，接近邊界時數值容易不穩定。於是實務上採用一個漂亮的技巧。

對數勝算比：把乘除變成加減

這個技巧是把機率改寫成對數勝算比（Log-odds）形式。勝算（odds）是「佔據機率 ÷ 空置機率」，再取對數就是對數勝算比。它的好處立竿見影：

- 更新變成加法：每來一筆觀測，只需把對應的「對數勝算比增量」加上去即可——乘除變成了加減，計算極快。
- 數值範圍無界：對數勝算比可以是任意實數（正無窮對應必定佔據、負無窮對應必定自由、0 對應五五波），不再卡在 0 到 1 的邊界，數值穩定。

- 需要時再轉回機率：平時都用對數勝算比累積，只在要顯示或決策時，才轉換回 0 到 1 的機率。

這個「取對數把乘除變加減」的手法，在工程與科學中隨處可見（例如分貝、pH 值），佔據格地圖只是其中一例。同學不必記憶公式，但要理解這個設計動機：讓「累積大量觀測」這件事變得又快又穩。

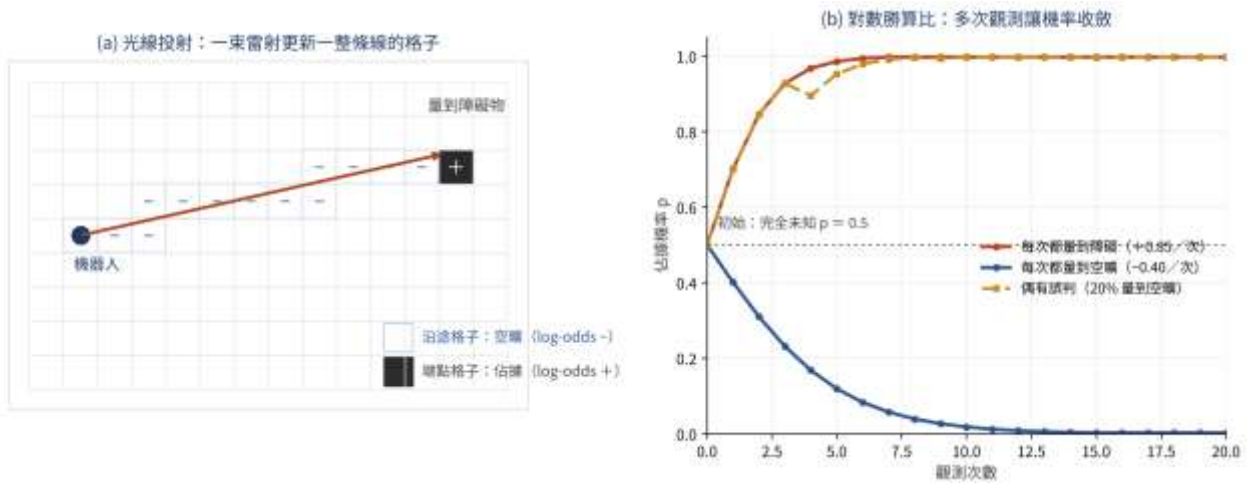
光線投射：一筆量測更新一整條線

這裡還有一個容易忽略但很重要的細節。當雷射量到「前方三公尺有牆」，它其實更新了一整條線上的格子，而不只是牆那一格：

- 雷射終點那一格（牆）：增加佔據機率（這裡有東西）。
- 雷射沿途穿過的所有格子（三公尺內）：降低佔據機率（雷射能穿過，表示這些地方是空的）。

這個「沿雷射路徑更新沿途格子」的過程稱為光線投射（Ray Casting），背後的模型稱為逆感測器模型（Inverse Sensor Model）——由一次量測反推沿途各格的佔據狀態。正是這個機制，讓機器人不只知道「哪裡有牆」，也知道「哪裡是確定可走的空間」，這對第 29 章的路徑規劃至關重要。

圖 13-1　光線投射與對數勝算比更新



左圖是光線投射的幾何。一束雷射量到「三公尺處有東西」，這句話其實帶來兩種資訊：端

點那一格有障礙物（加上正的 log-odds），而沿途經過的每一格都沒有東西擋住（各自加上負的 log-odds）。所以一束雷射不是只更新一格，而是更新一整條線——這也是為什麼機器人只要在房間裡走一圈，整個空曠區域就會被「掃」成白色。

右圖說明為什麼要用對數勝算比而不是直接存機率。紅線是某一格每次都被量到障礙的情形：第一次觀測後機率從 0.5 跳到約 0.70，三次後超過 0.90，五次後接近 0.99。藍線則是每次都被量到空曠，機率迅速趨近 0。

最值得注意的是黃色虛線：這一格大多數時候被量到障礙，但有 20% 的機會因為雜訊或反光而被誤判為空曠。可以看到曲線只是短暫下滑又被拉回去——單次的誤判無法推翻累積的證據。這正是機率式地圖相對於「看到就標黑」的根本優勢，也是第 11 章貝氏更新在二維格網上的具體實現。

> **為什麼要取對數**
>
> 貝氏更新本身是乘法：新的勝算 = 舊的勝算 × 量測的勝算。乘法做多了容易發生數值下溢，而且運算較慢。
>
> 取對數之後乘法變成加法，每次更新只需要一個整數或浮點數加法——這對每秒要更新數萬格的即時系統是決定性的差異。
>
> 還有一個隱藏好處：log-odds 可以直接設定上下限（例如 ±6），避免某一格因為被觀測上千次而「鎖死」，使地圖永遠無法反映環境變化。13.5 節的動態更新正是靠這個機制。

13.3　雷射掃描匹配

上一節假設「位姿已知」。但位姿到底怎麼估？除了里程計，光達本身就能提供強大的位姿線索——這就是掃描匹配（Scan Matching），2D SLAM 前端的核心技術。

核心想法：讓兩次掃描對齊

掃描匹配的想法非常直觀。機器人在位置 A 掃描一次，前進一小步到位置 B 再掃描一次。如果環境沒變（牆還是那面牆），那麼「B 的掃描」相對「A 的掃描」的位移與旋轉，就反映了機器人從 A 到 B 的移動。

換句話說：找出一個旋轉與平移，把 B 的掃描疊到 A 的掃描上，使兩者「最吻合」——這個旋轉與平移，就是機器人的相對移動。這是一個最佳化問題（第 12 章的精神）：調整位姿，讓兩次掃描的差異最小。

ICP：最經典的掃描匹配

最著名的掃描匹配演算法是 ICP（Iterative Closest Point，迭代最近點）。它的流程是一個反覆逼近的循環：

1. 對應（Correspondence）：對 B 掃描中的每個點，在 A 掃描中找最近的點，配成一對。
2. 求解（Alignment）：計算一個旋轉與平移，使所有配對點的距離總和最小。
3. 更新與迭代：套用這個變換，然後回到步驟 1 重新找對應點，反覆進行直到不再改善（收斂）。

ICP 的名字精準描述了它的運作：反覆（Iterative）地找最近點（Closest Point）並對齊。它簡單有效，是點雲配準的基石，第 17 章的 3D 建圖還會再遇到它。

掃描匹配的優點與陷阱

- 優點：光達精確，掃描匹配估出的相對移動，通常比里程計準得多，尤其在轉彎時不受輪子打滑影響。
- 陷阱一——環境重複：走廊、空曠大廳等「處處長得一樣」的場景，掃描匹配會找不到唯一的對齊方式（沿走廊滑動，掃描看起來都差不多），導致估計失準。這與第 9 章「白牆的詛咒」是同一類問題：缺乏獨特特徵。

- 陷阱二——初始猜測：ICP 需要一個好的初始猜測才容易收斂到正確解，否則可能陷入錯誤的局部最佳。這正是里程計派上用場的地方——下一節說明。

13.4 里程計與光達融合

第二篇反覆強調「感測器融合」，2D SLAM 是這個思想最乾淨的示範：里程計與光達的優缺點恰好互補，合起來遠勝任何一方單獨使用。

兩者的互補

特性	里程計（輪速）	光達掃描匹配
資料連續性	連續、高頻、平滑	受掃描頻率限制
短期精度	尚可	高（尤其轉彎）
長期行為	漂移累積	相對穩定
失效場景	打滑、地面不平	環境重複、特徵不足
需要初始猜測	不需要	需要（否則易發散）

表 13-1 里程計與光達掃描匹配的互補

融合的邏輯：里程計給起點，光達來校正

看出關鍵了嗎？里程計不需要初始猜測、且連續平滑，恰好能為「需要初始猜測」的掃描匹配提供起點；而掃描匹配精確、不漂移，恰好能校正里程計的漂移。兩者一拍即合：

4. 機器人移動一步，里程計先給出一個粗略的相對移動估計（初始猜測）。
5. 以這個猜測為起點，執行掃描匹配（ICP），精修出更準確的相對移動。
6. 用精修後的位姿更新地圖（13.2 節），並作為下一步的基礎。

這個「里程計預測、光達校正」的循環，正是第 11 章貝氏濾波「預測—更新」的具體化身：里程計負責預測（運動模型）、光達負責更新（觀測模型）。同學應該能感受到，第

三篇的抽象數學，在此變成了可操作的建圖流程。第 14 章的 Gmapping 與第 15 章的 Cartographer，都是這個基本循環的精緻化。

打滑時怎麼辦

當輪子打滑，里程計會突然給出錯誤的大位移。此時掃描匹配的價值更加凸顯：即使里程計說「我前進了半公尺」，只要光達掃描顯示「環境幾乎沒變」，掃描匹配就會把估計拉回「其實幾乎沒動」。這種「以環境觀測否決錯誤運動估計」的能力，正是 SLAM 比純里程計航位推算強韌的根本原因。

13.5 地圖解析度與更新

建 2D 地圖時，一個必須做的關鍵決定是：每個格子代表多大的實際範圍？這就是地圖解析度（Map Resolution），它牽動地圖品質、記憶體與運算的三方權衡。

解析度的取捨

地圖解析度指每個格子的實際邊長，室內常用 5 公分。這個選擇是一個典型的工程權衡：

- 解析度高（格子小，如 1 公分）：地圖細膩，能呈現細小結構（桌腳、門框），但記憶體用量暴增（格子數與邊長平方成反比——邊長減半，格子數變四倍），運算也變慢。
- 解析度低（格子大，如 20 公分）：記憶體省、運算快，但地圖粗糙，細小障礙可能被整格「抹平」，牆的位置也不精確。

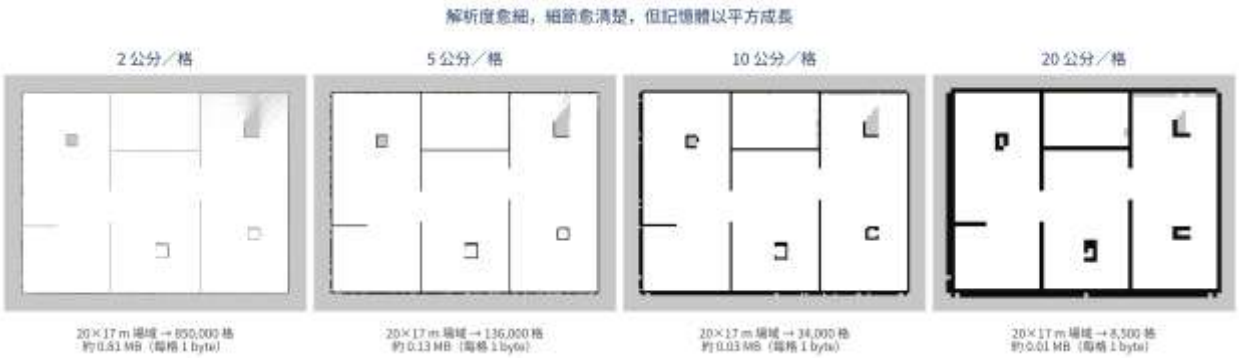
如何選？取決於應用需求。掃地機器人不需分辨一公分的細節，5 公分綽綽有餘；而需要精密對接的機器人，可能需要更高解析度。這呼應第 8 章光達角解析度的討論——地圖解析度再高，也不可能超越感測器本身的解析能力，兩者要匹配。

記憶體的現實

讓同學對記憶體用量有具體概念：一個 20 公尺 × 20 公尺的房間，用 5 公分解析度，就

是 400 × 400 = 16 萬個格子。若解析度提高到 1 公分，則變成 2000 × 2000 = 400 萬個格子——整整 25 倍。對大範圍場景（整棟大樓），這個成長會很可觀，這也是第 3 章提到 3D 地圖要用八叉樹（OctoMap）壓縮的原因。2D 佔據格因為只有平面，記憶體壓力相對小得多，這也是它適合教學與掃地機器人的原因之一。

圖 13-2　解析度的取捨：同一個場域的四種格子大小



這四張圖是同一趟掃描、同一份資料，只改變格子大小。2 公分的地圖牆面細緻、家具輪廓清楚；20 公分的地圖牆變成粗黑的塊，桌腳與椅子則整個糊成一團，甚至連門口的開口都快要被填平。

但代價寫在每張圖下面。以 20×17 公尺的場域計算，2 公分需要 85 萬格、約 0.81 MB；5 公分降到 13.6 萬格；10 公分只剩 3.4 萬格；20 公分則是 8,500 格。格子邊長減半，格數變四倍——記憶體以平方成長，而更新所需的運算量也同步增加。

實務上的常見選擇是 5 公分，這不是巧合：它剛好能分辨出一般門框（約 80 至 90 公分）、走道（約 1 公尺）與桌腳之間的間隙，而記憶體又還在嵌入式平台能負擔的範圍。掃地機器人為了省記憶體常用 5 至 10 公分；需要穿越狹窄通道的倉儲 AGV 則會用到 2 至 3 公分。

選型的方法與第 8.4 節判斷光達角解析度完全相同：先問「我最小要分辨多寬的空隙」，再回推格子大小。而且兩者必須搭配——用 2 公分的格子去存 1 度角解析度的光達資料，只會得到一張充滿空洞的地圖，因為感測器根本提供不了那麼細的資訊。

動態更新：地圖不是拍一張照就好

地圖需要持續更新，原因有二：

- 探索新區域：機器人走到沒去過的地方，把「未知」的灰色格子填成「自由」或「佔據」。
- 修正舊估計：反覆觀測同一區域，累積證據修正之前不確定的格子（13.2 節的貝氏更新）。

但這帶來一個微妙問題：如果環境本身變了怎麼辦？例如有人搬動了椅子。若地圖對舊觀測「記憶」太深（對數勝算比累積太高），就很難更新成新狀態；若記憶太淺，又容易被雜訊干擾。實務上會對對數勝算比設上下限（clamping），讓地圖保持一定的「可塑性」，能反映環境變化。這是第 16 章掃地機器人「動態環境」討論的伏筆。

13.6 走廊、教室、辦公室的 2D 建圖

理論講完，來看真實室內場景。不同的空間結構，對 2D 建圖有截然不同的挑戰。理解這些，同學才能預判「哪裡容易出問題」，這是實作前的重要準備。

教室：2D SLAM 的友善場景

一般教室是 2D SLAM 最理想的場景：四面牆界線分明、有桌椅提供豐富的幾何特徵、空間封閉不大。掃描匹配在這裡能穩定運作，建出的地圖清晰。這也是為什麼本書的實作活動多以教室為場域（第 14、29 章）——它讓同學在最順利的條件下先建立成功經驗。

走廊：掃描匹配的惡夢

長直走廊是 2D SLAM 的經典難題，正是 13.3 節「環境重複」陷阱的典型。走廊兩側是平行的長牆，沿走廊方向移動時，雷射掃描看起來幾乎不變——掃描匹配無法判斷「沿走廊走了多遠」（這個方向缺乏約束），只能確定「有沒有偏離走廊中線」（垂直方向有牆約束）。

這正是第 11 章 11.2 節那個「沿走廊方向拉長的不確定性橢圓」的來源！此時里程計的

價值凸顯——它能提供沿走廊方向的移動估計，補上掃描匹配缺失的約束。這是里程計與光達融合（13.4 節）不可或缺的又一個實例。長走廊建圖若只靠光達，幾乎必定失敗。

辦公室：動態與玻璃的挑戰

開放式辦公室綜合了多種挑戰：隔間玻璃讓光達「看穿」或產生錯誤反射（第 8 章的玻璃問題）、人員走動造成動態干擾、相似的格位排列容易混淆。這類環境往往需要更精細的參數調校，或搭配其他感測器。

大廳與中庭：空曠的困境

大型空曠空間（如挑高大廳）的問題是：光達的量測範圍有限（教學型約 8 至 12 公尺，第 8 章），當空間大於量測範圍，機器人走到中央時可能「四面都掃不到牆」，完全失去參考。此時掃描匹配無從進行，只能倚賴里程計硬撐，誤差快速累積。

場景	主要挑戰	關鍵對策
教室	幾乎無（理想場景）	直接建圖即可
長走廊	環境重複，沿走廊方向無約束	倚賴里程計補足縱向估計
辦公室	玻璃、動態人員、格位相似	參數調校、多感測器
空曠大廳	超出光達量程，四周掃不到	里程計硬撐、加大量程或加地標

表 13-2　典型室內場景的 2D 建圖挑戰

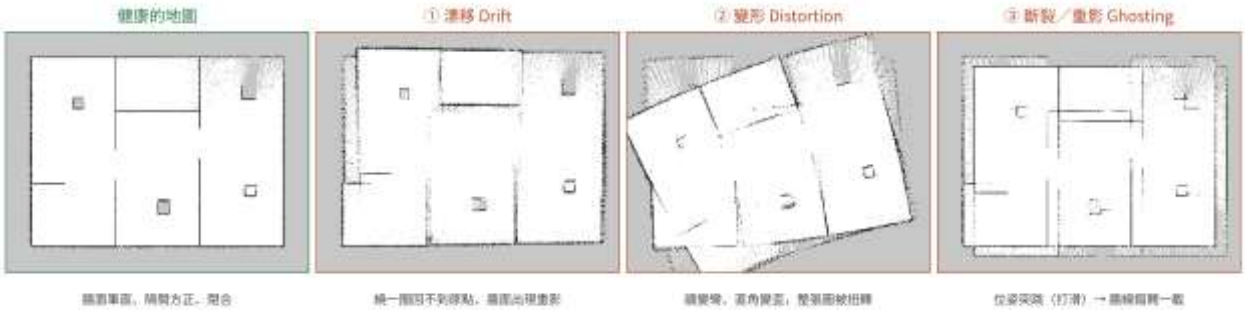
13.7　地圖品質評估：變形、斷裂、漂移

建完地圖，怎麼判斷它「建得好不好」？這是實作中極重要卻常被忽略的一環。本節教同學辨識三種常見的地圖病徵，以及如何評估品質——這也是第 31 章專題評分 Rubric 的基礎。

三種常見的地圖病徵

- 漂移（Drift）：最常見的問題。累積誤差使地圖逐漸偏移，最典型的症狀是「繞一圈回不到原點」——本該閉合的迴路，終點與起點錯開。這正是第 12 章閉環要解決的問題，也是判斷「需不需要閉環偵測」的依據。
- 變形（Distortion）：地圖的幾何被扭曲，直牆變成彎的、直角變成歪的、房間比例失真。多半來自掃描匹配的系統性誤差或里程計的尺度誤差（輪徑校正不準）。
- 斷裂/重影（Ghosting）：同一面牆在地圖上出現兩道（重影），或牆線斷開。通常是位姿估計在某處突然跳錯，導致同一結構被畫在兩個位置。走廊、空曠處最容易發生。

圖 13-3　三種常見的地圖病徵



這四張圖同樣來自同一份感測資料，差別只在位姿。最左邊是位姿正確時該有的樣子：牆面筆直、隔間方正、外框閉合。

① 漂移 Drift：位姿緩慢而持續地偏移，使同一面牆在不同時間被畫在不同位置，最終呈現為兩條平行的線；繞一圈回到原處時，起點與終點也對不上。這是最典型的「沒有閉環」症狀——機器人回來了，卻不知道自己回來了。

② 變形 Distortion：位姿帶有持續的角度偏差，整張地圖被漸進地扭轉。症狀是「每一小段看起來都正常，但整體歪掉」，走廊不再是直的、房間變成菱形。原因通常是輪徑或輪距校正不準（第 29 章會實際校正），或掃描匹配在長走廊中失效。

③ 斷裂 / 重影 Ghosting：位姿在某個瞬間突然跳動——機器人打滑、被人推了一下、或掃描匹配收斂到錯誤的解。症狀很好認：牆面在某個位置整齊地錯開一截，彷彿地圖被剪開後

沒對準就黏回去。

從病徵反推病因

看到漂移（牆面重影、起終點對不上）→ 閉環沒有被偵測到，或被偵測到卻沒有被接受。第 15 章的分支定界與第 22 章的視覺詞袋就是在解決這件事。

看到變形（牆彎、直角歪）→ 先懷疑里程計的系統性偏差（輪徑 / 輪距），再懷疑長走廊的掃描匹配退化。

看到斷裂（牆線整齊錯開一截）→ 找那個時間點發生了什麼：打滑、碰撞、或感測器資料掉幀。

這套「看症狀、猜病因」的能力，在第 29 章做實體建圖時會天天用到——而它需要的正是一雙看過正常地圖長什麼樣的眼睛。

如何評估地圖品質

評估方法分主觀與客觀兩類：

- 目視檢查（定性）：牆是否筆直？直角是否方正？迴路是否閉合？有無重影與斷裂？這是最快的第一步，有經驗的人一眼就能看出地圖好壞。
- 對照實測（定量）：量測地圖上幾個關鍵距離（走廊長度、房間寬度），與捲尺實測值比較，計算誤差百分比。這是客觀量化地圖精度的方法。
- 閉環一致性：讓機器人繞迴路回到起點，量測地圖上「起點」與「終點」的落差。落差越小，表示漂移控制越好。
- 與真值比對（研究級）：在有動作捕捉系統或已知精確地圖的環境，把估計地圖與真值疊合比對（第 31 章專題會用到）。

養成「建完圖就評估品質」的習慣至關重要。一張看似完整、實則變形的地圖，會讓後續的路徑規劃（第 29 章）全盤出錯——機器人會照著一張錯誤的地圖撞牆。這呼應第 8 章「感測器限制直接變成系統失敗」的觀念：地圖品質，直接決定整個導航系統的成敗。

至此，同學已經理解 2D 地圖「是什麼、怎麼建、怎麼評估」。但本章刻意把「建圖」與「定位」分開，假設位姿已知。真實的 SLAM 必須同時解這兩者——下一章 Gmapping 登場，它用第 11 章的粒子濾波，讓每個粒子各自攜帶一張地圖，一邊定位一邊建圖，漂亮地解開這個雞生蛋問題。理論與演算法，終於要合而為一了。

公式或流程

一、對數勝算比更新

某格子的佔據機率 p，其對數勝算比定義為：

$$l = \log(p \;/\; (1 - p))$$

每來一筆觀測，新的對數勝算比只需做加法：

$$l_new = l_old + l_measurement$$

其中 l_measurement 是本次觀測帶來的證據（佔據觀測為正、自由觀測為負）。這就是 13.2 節「把乘除變加減」的數學核心。需要顯示或決策時，再轉回機率：

$$p = 1 - 1 \;/\; (1 + \exp(l))$$

l 為正表示傾向佔據、為負表示傾向自由、為零表示五五波。實務上對 l 設上下限，保持地圖對環境變化的可塑性。

二、掃描匹配的最佳化目標

掃描匹配求一個相對位姿 T（旋轉 + 平移），使當前掃描 S 對齊參考掃描 R 後的總誤差最小：

$$T^* = \operatorname{argmin} \Sigma_i \,\| T \cdot s_i - r_{j(i)} \|^2$$

其中 s_i 是當前掃描的點、$r_{j(i)}$ 是參考掃描中與之對應的最近點。這是一個最小平方問題（第 12 章），ICP 以「找對應—求解—迭代」反覆逼近其解。

三、里程計與光達融合建圖流程

圖 13-4　2D SLAM 單步建圖流程（本章版本，假設閉環另行處理）

① 機器人移動一步

↓

② 里程計估出粗略相對移動（初始猜測）

↓

③ 以初始猜測為起點，執行掃描匹配（ICP）精修相對移動

↓

④ 更新機器人位姿（累加精修後的相對移動）

↓

⑤ 光線投射：沿雷射路徑，以對數勝算比更新沿途格子

↓

⑥ 回到 ①，處理下一步（週期性檢查閉環 → 第 14、15 章）

四、地圖記憶體估算

一張佔據格地圖的格子總數為：

$$\text{格子數} = (\text{寬} / \text{解析度}) \times (\text{高} / \text{解析度})$$

例如 20 m × 20 m 房間、解析度 0.05 m：格子數 = 400 × 400 = 160,000 格。解析度減半（0.025 m）則格子數變四倍（640,000 格）——這條式子量化了 13.5 節「解析度與記憶體」的平方關係。

13.8　2D 建圖的當代實務：從一次性地圖到終身地圖

佔據格地圖的數學在 1980 年代就定型了，本章介紹的對數勝算比更新至今沒有改變。但「地圖怎麼被使用」這件事，近十年有了相當大的轉變，而這個轉變直接影響同學在第 29 章會用到的工具。

一、從靜態圖檔到可持續更新的位姿圖

傳統做法是：建圖一次，存成一張 PGM 圖檔加一份 YAML 設定，之後導航時就載入這張靜態地圖來定位。這個流程簡單，但有個明顯的問題——環境會變。桌椅移位、貨架調整、隔間重做，地圖就過時了，只能整個重新建一次。

ROS 2 生態目前的主流工具 slam_toolbox 提出了不同的做法：它不只存地圖，而是把整個位姿圖（第 12 章的節點與邊）序列化存下來。下次開機時載入這個位姿圖，機器人可以選擇繼續建圖、只做定位、或是把新舊地圖合併。這稱為終身建圖（Lifelong Mapping）。

幾個值得知道的規格：其同步模式在約 2,800 平方公尺（三萬平方英尺）的建築中可達五倍以上的即時速度，約 5,600 平方公尺時仍有三倍；已知最大的同步建圖案例約為 18,600 平方公尺。非同步模式則適用於更大的空間，代價是會選擇性丟棄來不及處理的掃描。底層的最佳化求解器預設使用 Ceres，也可切換為 g2o——正是第 12.6 節介紹的那幾套。

二、定位也可以不用靜態地圖

slam_toolbox 還提供一種稱為彈性位姿圖定位的模式：載入序列化的地圖後，把進來的掃描與最近一段時間的掃描緩衝區做匹配，而不去永久改動底層地圖。它同樣接受 RViz 的初始位姿設定，使用方式與第 16 章要介紹的 AMCL 相似。

對同學的意義是：第 16 章的 AMCL（粒子濾波定位）不再是唯一選項。不過工具的作者本人建議初學者仍從 AMCL 入手——因為它的行為直觀、資料豐富、出問題時容易診斷。本書的安排也依循這個建議。

三、多張地圖的合併

還有一個實務上非常有用的能力：把多次建圖的結果合併成一張全域一致的地圖。大型場域

很難一次掃完——電池會沒電、某些區域暫時進不去——於是分次建圖、事後合併就成了標準做法。合併可以在 RViz 中手動拖曳對齊，也可以用運動學工具自動完成。

這件事背後的數學就是第 12 章的位姿圖最佳化：把兩張圖的位姿圖接在一起，加上幾條人工指定的「這兩個節點是同一個地方」的邊，再重新求解一次。理解了第 12 章，這個功能就不再神祕。

四、佔據格之外

最後值得一提的是，佔據格不是唯一的二維地圖表示。近年在導航堆疊中逐漸普及的還有幾種：代價地圖（Costmap）把障礙物向外膨脹一圈，直接存「這裡能不能走」而非「這裡有沒有東西」；距離場（ESDF）每一格存到最近障礙物的距離，讓避障與路徑平滑化的計算更有效率；語意圖層則在幾何之上疊加「這是玻璃」「這是樓梯」「這是禁區」等資訊——第 3.6 節提過的 Nav2 圖層式代價地圖正是這個架構。

但這些表示法幾乎都建立在佔據格之上，或與它並存。所以本章學到的仍然是地基：理解了「每一格存一個機率、以對數勝算比累積證據」，其餘的變體都只是換一種存法。

> **給同學的三個提醒**
>
> 一、地圖是會過時的。任何打算長期運行的機器人都必須處理地圖更新，而不是假設環境永遠不變。
>
> 二、存位姿圖比存圖檔更有價值。圖檔只能看，位姿圖可以繼續最佳化、可以合併、可以回頭修正。
>
> 三、本章的簡化（位姿已知）會在下一章被解除。第 14 章開始，位姿與地圖要一起估計——那才是 SLAM 真正的樣子。

案例

案例 13-1 掃地機器人 App 上那張地圖：佔據格的日常

打開任何一款支援地圖功能的掃地機器人 App，那張你用來設定禁區、指定房間的房間地圖，底層就是本章的佔據格地圖。黑色的牆線是高佔據機率的格子、白色的可清掃區是低佔據機率的自由空間、而機器人還沒去過的地方則是空白（未知）。

觀察這張地圖的「成長過程」很有教育意義：機器人第一次清掃時，地圖是一點一點「長」出來的——走到哪、掃到哪、畫到哪，未知的區域逐漸被填滿。這正是 13.5 節「動態更新—探索新區域」的視覺化。若機器人在某處打滑或撞到黑色家具（光達看不到，第 8 章），你可能會在地圖上看到牆線歪斜或缺口——這就是 13.7 節的漂移與斷裂，活生生出現在你家客廳。

更進階的觀察：有些掃地機器人第二次清掃同一個家時，地圖會比第一次更精確、更平整。這是因為它累積了更多觀測，對每一格的估計更有把握（13.2 節的貝氏累積），甚至可能執行了閉環修正（第 12 章）。一台掃地機器人，其實是本篇所有理論的隨身教材。

案例 13-2 長走廊翻車記：一個真實的教學失敗

大學課堂上，一個幾乎年年上演的場景：同學在教室裡把 2D SLAM 建圖跑得很順，信心滿滿地把機器人開進長廊，結果地圖瞬間「爆掉」——走廊被畫得忽長忽短、牆線扭曲重疊，機器人徹底迷失。

原因就是 13.3 與 13.6 節反覆強調的「環境重複」：長走廊沿縱向缺乏獨特特徵，掃描匹配無法確定「沿走廊走了多遠」。若此時里程計又因地面材質改變（磁磚換成地毯）而不準，兩個資訊都失效，建圖必然失敗。

這個「失敗」其實是最好的一堂課。它讓同學親身體會：SLAM 不是萬能的，環境結構會直接決定成敗。解法呼應全書：融合可靠的里程計（13.4 節）、加大光達量程看到走廊盡頭、或在走廊貼 AprilTag（第 4 章）提供獨特地標。理解為什麼失敗，比在教室裡順利建圖學到的更多——這也是本書為何在每一章都誠實討論「限制與失敗」的原因。

案例 13-3 從 2D 到智慧：掃地機器人的演進

回顧掃地機器人的演進，是一部 2D SLAM 的普及史。最早的機型（2002 年前後）根本沒有地圖，採「隨機碰撞」——撞到就轉彎，清掃路線雜亂、重複又遺漏（第3、16章）。2010 年 Neato 首度把 2D 光達裝上掃地機，讓它能建佔據格地圖、規劃系統性路徑。2017 年小米以親民價格普及光達掃地機（第 1 章案例），讓佔據格地圖進入千家萬戶。

近年的機型更進一步：加入 3D ToF 相機辨識並繞開電線、鞋子、寵物排泄物（彌補 2D 光達的高度盲區，第 8 章）；用 AI 辨識房間類型自動分區；甚至記住多層樓的地圖。這條演進線清楚顯示：2D 佔據格地圖是基礎，但真實產品會不斷疊加感測器與 AI 來彌補單一技術的限制——這正是全書貫穿的融合思想，也預告了第九篇 AI 語意導航的方向。同學今天學的 2D 建圖，是理解這一切的起點。

實作活動

實作 13-1 親手建一張佔據格地圖

目的：透過程式從零實作佔據格地圖的更新，親身理解對數勝算比、光線投射與掃描匹配。本實作以 Python 為主，分模擬與資料兩條路徑，是第 14 章 Gmapping 實作的直接暖身。

第一部分：對數勝算比更新（全體）

1. 用 numpy 建立一個二維陣列當作地圖（例如 200×200 格，每格初始對數勝算比為 0，代表未知）。
2. 模擬單一雷射光束：給定機器人位置、光束方向與量到的距離，實作光線投射——把沿途格子的對數勝算比減少（自由）、終點格子增加（佔據）。
3. 把對數勝算比轉回機率，用 matplotlib 以灰階顯示地圖（黑=佔據、白=自由、灰=未知），確認單束雷射畫出一條「白線到黑點」。
4. 模擬一整圈掃描（360 束）：讓機器人在房間中央，對四周的模擬牆面各發一束，觀察是否畫出房間輪廓。

5. 多次觀測：讓機器人在同一位置重複掃描數次（加入隨機雜訊），觀察牆線如何隨觀測次數增加而變得更「確定」（對數勝算比累積，顏色更黑）。

第二部分：移動建圖（全體或大學部）

6. 設計一個簡單的模擬房間（用程式定義幾道牆），讓一台虛擬機器人沿預定路線移動，每步用已知位姿與模擬光達更新地圖，觀察完整房間地圖如何逐步「長」出來。
7. 加入里程計雜訊：故意讓機器人的位姿估計帶有累積誤差（漂移），觀察建出的地圖如何變形、牆線如何重影——親身重現 13.7 節的病徵。
8. 解析度實驗：把同一場景分別用 2 公分、5 公分、20 公分解析度建圖，比較地圖細膩度、記憶體用量（格子數）與運算時間。

第三部分：ROS 真實/模擬建圖（有 ROS 環境者）

9. 在 TurtleBot3 模擬（第 28 章）中，啟動內建的 2D SLAM（如 slam_toolbox 或 gmapping），遙控機器人在模擬教室走一圈，在 RViz 中即時觀察佔據格地圖的成長。
10. 把機器人開進模擬長走廊，觀察本章 13.6 節描述的「走廊困境」是否出現（地圖沿走廊方向變形）。
11. 建完圖後，用 13.7 節的方法評估品質：目視檢查牆是否筆直、量測地圖上的房間尺寸與模擬設定值比較。

討論問題

1. 第一部分中，同一面牆被觀測 1 次與觀測 10 次，地圖上的顏色（佔據機率）有何差異？這說明對數勝算比累積的什麼作用？
2. 第二部分加入里程計漂移後，地圖出現了哪一種病徵（變形/斷裂/漂移）？若要修正它，你會用第 12 章的什麼技術？
3. 解析度從 5 公分改為 2 公分，格子數增加了幾倍？記憶體與運算時間的變化符合 13.5 節的平方關係嗎？

4. （有 ROS 者）長走廊的地圖出了什麼問題？對照 13.6 節，你認為加入里程計、加大光達量程、貼 AprilTag，哪個對策最有效？為什麼？

延伸挑戰（大學部）

實作一個簡化版的 ICP 掃描匹配：給定兩次帶有已知相對位移的模擬掃描，用「找最近對應點—求最佳旋轉平移—迭代」的流程，估計出這個相對位移，並與真值比較。進一步在「長走廊」情境（縱向特徵不足）測試你的 ICP，觀察它為何沿走廊方向估計失準——這將讓你深刻理解為何 Gmapping（第 14 章）要用粒子濾波，而非單純倚賴掃描匹配。

延伸閱讀

1. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（第 9 章 Occupancy Grid Mapping 為本章的權威依據）
2. Cyrill Stachniss, SLAM 課程之 Occupancy Grid Maps、Scan Matching、ICP 系列影片（YouTube）.（以動畫講解本章核心，自學首選）
3. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992.（ICP 掃描匹配的原始論文）
4. A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, 1989.（佔據格地圖的開創性論文）
5. slam_toolbox 開源專案（GitHub）與 ROS 官方文件.（現代 ROS 2 主流的 2D SLAM 工具，取代舊 gmapping，實作首選）
6. ROBOTIS TurtleBot3 e-Manual 之 SLAM 章節（emanual.robotis.com）.（2D 建圖的完整實作步驟）
7. 高翔、張濤，《視覺 SLAM 十四講》之地圖相關章節.（雖以視覺為主，對地圖表示法的討論可與本章互補）

從已知位姿到同時估計：本章的位置回顧

本章做完了 SLAM 的後半段：給定位姿，把感測資料變成地圖。這一半其實相當直接——光線投射加上對數勝算比，不到一百行程式就能寫完。

困難的是另一半。本章從頭到尾假設位姿是已知且正確的，而圖 13-3 已經讓同學看到，這個假設一旦破裂，地圖會壞成什麼樣子。真實的機器人沒有人告訴它自己在哪裡——它只有一個會漂移的里程計，以及本章 13.3 節的掃描匹配。

掃描匹配其實已經是 SLAM 的雛形了：它用「把這次掃描對齊到地圖」的方式反過來修正位姿。但它有兩個弱點。其一，它只看局部——在 13.6 節的長走廊中，沿走廊方向怎麼滑都對得上，匹配無從判斷自己走了多遠。其二，它只有一個答案——一旦匹配錯了，沒有任何機制能回頭。

第 14 章的 Gmapping 用第 11 章的粒子濾波解決第一個弱點：同時保留數十個位姿假設，讓觀測逐一淘汰不合理的。第 15 章的 Cartographer 則用第 12 章的位姿圖最佳化解決第二個弱點：保留所有歷史，閉環時一次修正。兩章一起讀，就是第 11 章與第 12 章那兩條路線在同一個問題上的正面對決——而評判的標準，就是本章 13.7 節教的那套地圖品質判準。

小結

1. 佔據格地圖把平面切成方格，每格記錄佔據機率（佔據/自由/未知三態）；「未知」不等於「自由」，這是它優於二元地圖之處。建圖的前提是已知位姿，本章暫將建圖與定位分開，專注於「有位姿如何建圖」。
2. 佔據機率隨觀測以貝氏方式累積更新；實務用對數勝算比，把乘除變成加減、數值範圍無界又穩定；一筆雷射量測透過光線投射，更新終點格（佔據）與沿途格（自由）一整條線。
3. 掃描匹配比對前後兩次雷射掃描估計相對移動，經典演算法為 ICP（找對應—求解—迭代）；它精確但怕環境重複（走廊）且需要好的初始猜測。

4. 里程計與光達恰好互補：里程計連續、免初始猜測但漂移，光達精確、不漂移但需初始猜測且怕重複環境；融合邏輯是「里程計預測、光達校正」，正是貝氏濾波預測—更新的化身，並能以環境觀測否決打滑造成的錯誤運動。
5. 地圖解析度是品質、記憶體、運算的三方權衡，格子數與解析度成平方反比；地圖需動態更新以探索新區並修正舊估計，並對對數勝算比設限以保持對環境變化的可塑性。
6. 不同場景挑戰各異：教室理想、長走廊因縱向無約束而最難（需里程計補足）、辦公室有玻璃與動態、空曠大廳超出光達量程；預判挑戰是實作成功的關鍵。
7. 地圖三大病徵為漂移（繞圈回不到原點）、變形（幾何扭曲）、斷裂重影（同牆兩線）；評估方法有目視檢查、對照實測、閉環一致性與真值比對。地圖品質直接決定導航成敗。

本章刻意把建圖與定位分開，假設位姿已知。但真正的 SLAM 必須同時解開這兩者——這正是第 1 章的雞生蛋問題。下一章 Gmapping 登場：它運用第 11 章的粒子濾波，讓每一個粒子各自攜帶一條軌跡與一張地圖，在眾多假設中「適者生存」，漂亮地同時完成定位與建圖。第三篇的粒子濾波理論，即將在第一個完整的 SLAM 系統中大顯身手。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 14 章

Gmapping：粒子濾波 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第 13 章建了一張地圖，但它靠的是一個不成立的假設——位姿已知。本章要把這個假設拿掉，正面迎戰第 1 章提出的雞生蛋問題。

Gmapping 給的答案是第 11 章的粒子濾波，但做了一個關鍵的轉折：不要讓粒子去表示「位姿與地圖」的聯合分布，而是讓每一顆粒子只負責一條位姿軌跡，地圖則由該粒子自己依第 13 章的方法建出來。這個拆解稱為 Rao-Blackwell 化，是本章的核心。

本章與第 15 章是一組對照實驗：同一個問題（2D 光達 SLAM），Gmapping 走第 11 章的濾波路線，Cartographer 走第 12 章的最佳化路線。讀完這兩章再回頭看第 12.7 節的「濾波與圖優化總比較」，那張表就不再是抽象的條列。

圖 14-0　第 14 章與全書的連結地圖

【承接前章】

第 11 章　粒子濾波　→　本章的骨架（預測、加權、重採樣）

第 13 章　佔據格建圖　→　每顆粒子各自建一張

第 13.3 節　掃描匹配　→　本章改良提議分布的關鍵零件

【本章負責】

Rao-Blackwell 化：把聯合分布拆成「位姿用粒子、地圖用解析」

粒子數、權重、重採樣與粒子退化

Gmapping 兩項關鍵改良：改良提議分布、選擇性重採樣

【往下延伸】

→ 第 15 章　Cartographer：同一問題的最佳化解法（對照組）

→ 第 16 章　AMCL：粒子濾波用於「已知地圖中定位」

→ 第 6.7 節　磁力計自動校正：同一套 RBPF 換個應用對象

→ 第 29 章　在實體小車上調 Gmapping 參數

學習目標

上一章我們假設「位姿已知」來建圖，但真正的 SLAM 必須同時解開定位與建圖。本章介紹第一個完整的 SLAM 系統——Gmapping。它運用第 11 章的粒子濾波，讓每個粒子各自攜帶一條軌跡與一張地圖，在眾多假設中「適者生存」，漂亮地解開第 1 章的雞生蛋問題。學完本章之後，同學應能夠達成下列目標：

7. 說明 Gmapping 的基本概念與它在 SLAM 發展史上的地位。
8. 解釋 Rao-Blackwellized 粒子濾波的核心思想：為何「每個粒子帶一張地圖」能解開雞生蛋問題。
9. 說明粒子數、權重與重採樣的作用，以及粒子退化與有效粒子數的概念。
10. 說明 Gmapping 兩項關鍵改良：改良提議分布與選擇性重採樣。
11. 說明在 ROS 中執行 Gmapping 建圖的完整流程與必要前置條件。
12. 辨識常見的建圖失敗案例——輪子打滑、場景重複、光達遮蔽——並提出對策。
13. 完成以教學小車或模擬器建立教室地圖的實作。

核心概念

中文名詞	英文名詞	概念說明
粒子濾波 SLAM	RBPF-SLAM	以粒子濾波同時估計軌跡與地圖的 SLAM 方法。
Rao-Blackwell 化	Rao-Blackwellization	把問題拆解，一部分用粒子取樣、另一部分用解析法求解。
粒子	Particle	一個完整的假設：包含一條軌跡與其對應的一張地圖。
權重	Weight	該粒子的假設與感測器觀測吻合的程度。
重採樣	Resampling	依權重複製高分粒子、淘汰低分粒子的過程。

中文名詞	英文名詞	概念說明
粒子退化	Particle Depletion	重採樣過度導致粒子多樣性喪失的現象。
有效粒子數	Effective Sample Size	衡量粒子多樣性的指標，用於決定是否重採樣。
提議分布	Proposal Distribution	產生下一步粒子候選位置的機率分布。
掃描匹配	Scan Matching	比對雷射掃描以精修位姿，Gmapping 用它改良提議分布。
選擇性重採樣	Selective Resampling	只在有效粒子數過低時才重採樣，避免退化。
地圖伺服器	map_server	ROS 中儲存與載入地圖檔的工具。
地圖檔	pgm / yaml	ROS 地圖的影像檔與描述檔格式。

14.1 Gmapping 的基本概念

Gmapping 是 2D 雷射 SLAM 領域最著名、也最廣泛使用的開源系統之一。它由 Grisetti、Stachniss 與 Burgard 於 2007 年提出——正是第 2 章介紹的德國 Freiburg / Bonn 機率機器人學研究群。同學讀完本章後，若去看 Stachniss 教授在 YouTube 上親自講解粒子濾波 SLAM 的影片，會有「原作者現身說法」的難得體驗。

Gmapping 解決什麼問題

第 13 章我們刻意假設「位姿已知」，專注於建圖。但真實機器人的位姿正是未知的——而且位姿錯了，地圖就跟著錯。Gmapping 要同時回答兩個互相依賴的問題：機器人走過的軌跡是什麼？環境地圖長什麼樣？

它的答案極具巧思：不要試圖找出「唯一正確」的軌跡，而是同時維持數十到數百個「可能的軌跡假設」，每一個假設都各自建一張地圖；然後看哪些假設與感測器觀測最吻合，讓它們存活繁衍，不吻合的則被淘汰。這正是第 11 章粒子濾波「適者生存」思想的直接應用。

輸入與輸出

- 輸入：2D 光達掃描（LaserScan，第 8 章）、里程計（第 3、13 章）、以及機器人的座標框架設定（TF，第 10 章）。
- 輸出：一張佔據格地圖（第 13 章）、以及機器人在地圖中的位姿（即 map→odom 的座標變換，第 10.7 節）。

注意這個輸入輸出清單：它幾乎把前面所有章節都用上了。這正是本篇的意義——理論在此匯流成一個可以真正跑起來的系統。

歷史地位

Gmapping 長年是 ROS 生態中 2D SLAM 的預設選擇，無數大學實驗室與教學平台以它入門。近年在 ROS 2 環境中，slam_toolbox 等基於圖優化（第 12 章）的方案逐漸成為

主流，但 Gmapping 的教學價值絲毫未減：它是理解「粒子濾波如何做 SLAM」最清晰的範例，也是連接第 11 章理論與真實系統的最佳橋樑。

14.2 Rao-Blackwellized Particle Filter

這是本章最核心的觀念。Gmapping 的全名是「Rao-Blackwellized 粒子濾波 SLAM」，這個拗口的名詞背後，是一個非常聰明的拆解。

直接用粒子濾波做 SLAM 的困境

回想第 11 章：粒子濾波用一群樣本近似機率分布。若直接套用到 SLAM，每個粒子必須同時代表「機器人位姿」與「整張地圖」的一個假設。問題是——地圖有數萬個格子，等於數萬個未知數。要用粒子在如此高維的空間中取樣，所需粒子數會是天文數字，完全不可行。這稱為維度詛咒（Curse of Dimensionality）。

Rao-Blackwell 化：聰明的拆解

Rao-Blackwell 化的核心洞見是：把問題拆成兩部分，一部分用粒子取樣，另一部分用解析方法直接算。

關鍵觀察在於第 13 章的結論：只要知道機器人的完整軌跡，建地圖就是一件確定性的簡單事（用對數勝算比更新即可，不需取樣）。換句話說，地圖並不需要「猜」，它可以由軌跡直接算出來。

於是拆解就很自然了：

- 用粒子取樣的部分：機器人的軌跡（只有每步三個數：x、y、θ）。這是低維的，粒子濾波應付得來。
- 用解析方法算的部分：地圖。每個粒子拿著自己那條軌跡，用第 13 章的方法直接算出對應的地圖。

觀念核心：每個粒子是一個「平行世界」

想像 100 個平行世界，每個世界裡的機器人走了略微不同的路線（軌跡假設），

因此各自畫出略微不同的地圖。

新的雷射掃描到來時，每個世界都問自己：「以我的地圖與位置，這筆掃描合理嗎？」

合理的世界得高分（權重高）、不合理的得低分。

重採樣時，高分世界被複製、低分世界消失。存活下來的，就是與觀測最一致的假設。

這就是 Gmapping：用平行世界的競爭，同時找出正確的軌跡與地圖。

圖 14-1　Rao-Blackwell 化：每顆粒子都帶著自己的地圖



這四張圖是用同一份光達資料跑出來的，差別只在四顆粒子各自假設的軌跡不同——每顆粒子的里程計誤差模型不一樣，於是走出四條不同的路徑，也就建出四張不同的地圖。

粒子 1 的假設最接近真實，建出的地圖牆面筆直、隔間方正；粒子 4 的假設偏差最大，地圖整個被扭轉、牆面模糊。而權重正是由「這次的雷射掃描與我這張地圖有多吻合」算出來的——權重 0.46、0.29、0.16、0.10，與地圖品質的順序完全一致。

這就是 Rao-Blackwell 化的全部精神。粒子不需要同時猜位姿和地圖（那是一個高到無法處理的維度），它只要猜一條軌跡；地圖是軌跡的必然結果，用第 13 章的方法算出來就好。於是粒子濾波從「不可能」變成「可行」。

請特別注意一件事：被淘汰的不只是錯誤的位姿，還有那張錯誤的地圖。重採樣時粒子 1 被複製多份、粒子 4 被刪除，連帶它那張歪掉的地圖也一起消失。這是 SLAM 與單純定位最大的不同——錯誤的假設會同時污染位姿與地圖，所以必須整組一起淘汰。

> **一個常被問的問題：記憶體夠嗎**
>
> 每顆粒子都帶一張地圖，聽起來很耗記憶體——確實如此。以 30 顆粒子、20×17 公尺場域、5 公分解析度計算，總格數是 30 × 136,000 = 408 萬格，約 4 MB。
>
> 這在筆電上毫無問題，但在只有數十 MB 記憶體的嵌入式平台上就是真實的限制。這也是為什麼 Gmapping 的粒子數通常只設數十而非數千。
>
> 實作上還有一個技巧：粒子重採樣時，被複製的粒子可以共用同一張地圖（寫入時才複製，Copy-on-Write），能大幅節省記憶體。

為什麼這樣就解開了雞生蛋問題

第 1 章的雞生蛋問題是：建圖需要位姿、定位需要地圖。RBPF 的解法是——不必先有誰。每個粒子先「假設」一條軌跡（不需要地圖就能假設），依此軌跡建圖（建圖只需軌跡），再用觀測評估這個假設好不好（評估需要地圖，而地圖剛剛建好了）。整個循環自洽，不需要任何一方先存在。這個優雅的拆解，正是 Gmapping 在 SLAM 史上的重要貢獻。

14.3 粒子數、權重與重採樣

本節深入 RBPF 的三個實務關鍵：要用幾個粒子？怎麼給分？什麼時候淘汰？

一、粒子數：精度與運算的權衡

粒子數是 Gmapping 最重要的參數。它的取捨很直觀：

- 粒子多（如 100 個）：能涵蓋更多假設，對複雜環境與較大不確定性更穩健，但每個粒子都要維護一張地圖，記憶體與運算成本高。
- 粒子少（如 5 至 30 個）：快速輕量，但假設涵蓋不足，遇到大幅漂移或環境重複時，可能所有粒子都「猜錯」而無法挽回。

教學實務上，一般教室與走廊環境用 30 個粒子多半足夠；若環境大或里程計品質差，可增加到 50 至 100 個。這裡有一個重要提醒：粒子數與地圖大小相乘決定記憶體用量——100 個粒子各帶一張 400×400 的地圖，就是 1600 萬個格子。這是 Gmapping 難以用於超大場景的根本原因，也是圖優化方法（第 15 章 Cartographer）後來居上的動機之一。

二、權重：用觀測給假設打分數

每個粒子的權重，代表「這個假設與最新觀測的吻合程度」。計算方式是：用該粒子的位姿與地圖，預測「應該會量到什麼」，再與實際雷射掃描比較。

若某粒子的地圖顯示「前方 3 公尺有牆」，而實際掃描也量到 3 公尺，兩者吻合，權重高；若某粒子的地圖說「前方是空的」，實際卻量到牆，顯然這個粒子的位姿假設偏了，權重低。這正是第 11 章觀測模型的具體實現。

三、重採樣：適者生存，以及它的陷阱

重採樣依權重複製高分粒子、淘汰低分粒子，讓粒子群逐漸聚集到正確的假設。但這裡藏著一個危險的陷阱：粒子退化（Particle Depletion）。

若每一步都重採樣，粒子的多樣性會迅速流失——最後所有粒子可能都是同一個祖先的複製品，彼此幾乎一樣。這時粒子濾波就失去了「維持多個假設」的意義。更糟的是：如果那個被大量複製的祖先其實猜錯了，系統就再也沒有其他假設可以挽救，SLAM 徹底失敗。

這個問題在 SLAM 中特別嚴重，因為錯誤的軌跡假設一旦獨佔，對應的錯誤地圖也就定案，無法回頭修正（這也是第 12 章指出濾波方法相對圖優化的弱點）。Gmapping 的第二項關鍵改良，正是為了對付它——見下一節。

14.4 Gmapping 的兩項關鍵改良

如果只是「粒子濾波加上每粒子一張地圖」，系統會因為需要大量粒子而不實用。Gmapping 之所以成功，關鍵在於兩項改良，它們讓所需粒子數大幅降低。理解這兩項改良，就理解了 Gmapping 的精髓。

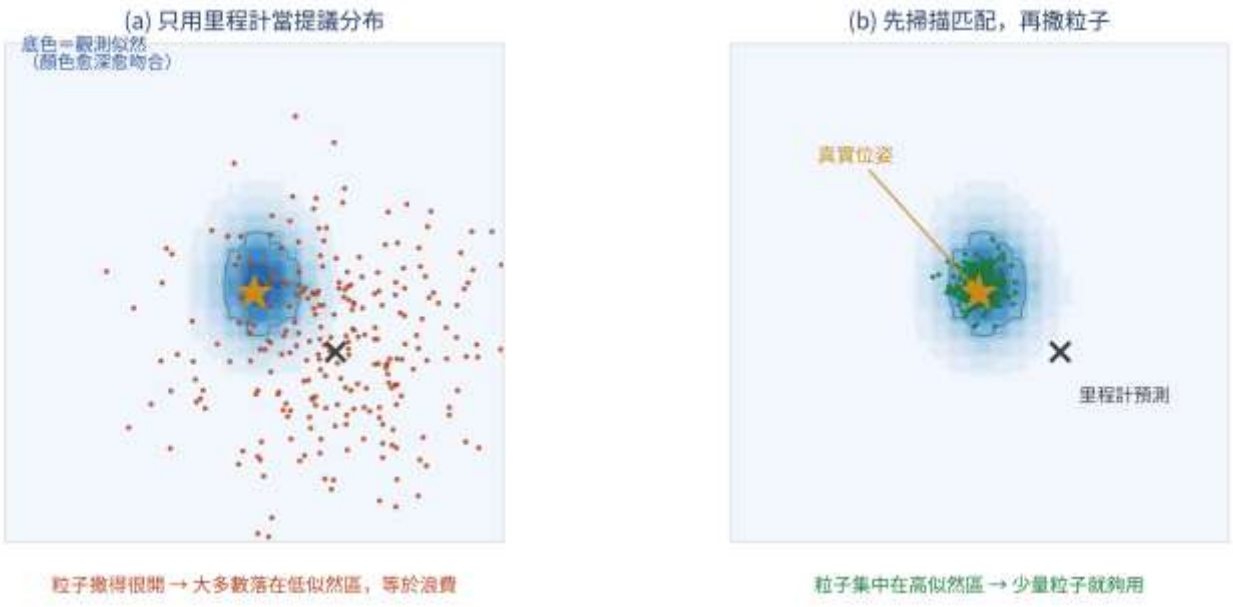
改良一：用掃描匹配改良提議分布

提議分布（Proposal Distribution）是指「下一步要把粒子撒在哪裡」的機率分布。最樸素的做法是只依里程計：粒子依里程計移動，再加上隨機雜訊散開。

問題是里程計雜訊往往很大，粒子會撒得很開，而真正正確的位姿只落在其中一小塊區域。大部分粒子被浪費在明顯錯誤的位置上，因此需要大量粒子才能「碰巧」有幾個落在正確處。

Gmapping 的改良是：先用第 13 章的掃描匹配精修位姿，再以精修後的結果為中心，在一個小得多的範圍內撒粒子。

圖 14-2 提議分布的改良



兩張圖的底色都是同一份觀測似然——顏色愈深，代表「機器人若在這裡，雷射掃描會與

地圖愈吻合」。金色星號是真實位姿，灰色叉號是里程計的預測位置，兩者之間有明顯的偏差。

左圖是樸素做法：以里程計預測為中心撒粒子。由於里程計雜訊大，粒子散布的範圍遠大於高似然區，絕大多數粒子落在幾乎不可能的位置上——它們會在加權時拿到接近零的權重，等於白算一輪。

右圖是 Gmapping 的做法：先用第 13.3 節的掃描匹配找出似然最高的位置，再以那裡為中心撒一小撮粒子。可以看到幾乎每一顆粒子都落在深色區域內，每一顆都是「有意義的假設」。

效果在數字上相當驚人：所需粒子數從數千降到數十。這不只是省運算，更是讓 Gmapping 能在當年的嵌入式電腦上即時運行的關鍵。同學若把這張圖與第 11 章圖 11-4 的粒子濾波四步驟對照，會發現改良的其實是第②步「預測」——把盲目的撒點換成有依據的撒點。

這個改良的效果驚人：它讓 Gmapping 用數十個粒子就能達到樸素方法需要數千個粒子的效果。同學應該能看出，這正是第 13.4 節「里程計預測、光達校正」思想的延伸應用。

改良二：選擇性重採樣

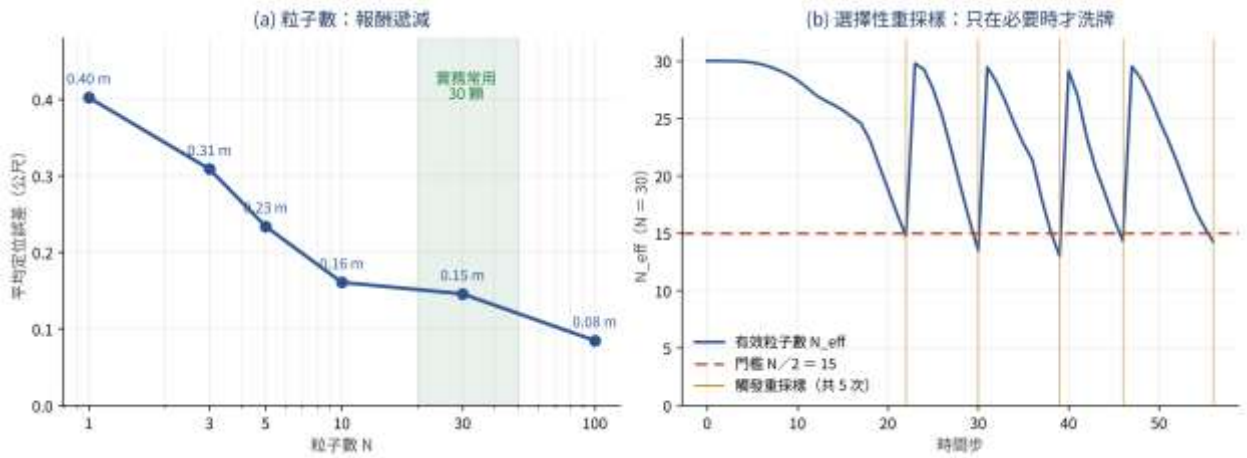
針對 14.3 節的粒子退化問題，Gmapping 採用選擇性重採樣（Selective Resampling）：不是每步都重採樣，而是先評估粒子的多樣性，只在多樣性太低時才重採樣。

衡量多樣性的指標稱為有效粒子數（Effective Sample Size,ESS）。若所有粒子權重相近（表示大家都差不多合理），ESS 高、多樣性好，不需重採樣；若少數粒子權重極高而其餘極低（表示已經分出勝負），ESS 低，此時才重採樣以集中資源。

這個策略在「保持多樣性」與「集中到正確假設」之間取得平衡，大幅減緩了粒子退化。這兩項改良——改良提議分布降低所需粒子數、選擇性重採樣避免退化——正是

Gmapping 論文的核心貢獻，也是它能成為實用系統的原因。

圖 14-3　粒子數的取捨與選擇性重採樣



左圖是實際跑出來的結果：同一條軌跡、同一組雜訊設定，只改變粒子數。1 顆粒子時平均定位誤差約 0.40 公尺——這其實就是沒有濾波、只靠單一假設推算；加到 5 顆降為 0.23 公尺；10 顆是 0.16 公尺；30 顆是 0.15 公尺；而 100 顆也只降到 0.08 公尺。

請注意 10 顆到 30 顆之間幾乎沒有改善，而 30 顆到 100 顆（運算量增加三倍以上）只換來 0.07 公尺。這就是報酬遞減——也正是案例 14-2 所說「三十個粒子夠用又不夠用」的量化版本。實務上 Gmapping 的預設粒子數就落在這個甜蜜點附近。

右圖展示選擇性重採樣的運作。藍線是有效粒子數 N_eff 隨時間的變化：剛重採樣完所有權重相等，N_eff 等於粒子總數 30；隨著觀測累積，權重逐漸集中到少數粒子身上，N_eff 一路下滑。當它跌破門檻 N / 2 = 15（紅色虛線）時才觸發重採樣（黃色豎線），整段軌跡共觸發 5 次。

這條鋸齒曲線把 14.3 節的粒子退化講得很清楚：若每一步都重採樣，多樣性會被反覆摧毀，正確的假設可能在還來不及證明自己之前就被刪掉；若從不重採樣，則所有權重都集中到一顆粒子，其餘全成了擺設。選擇性重採樣就是在這兩個極端之間取平衡。

14.5　ROS Gmapping 實作流程

理論講完，本節說明在 ROS 中實際執行 Gmapping 建圖的完整流程。這也是第 29 章實作章節的預習。

前置條件：三樣東西缺一不可

許多同學第一次跑 Gmapping 失敗，原因幾乎都出在前置條件沒備齊：

7. 雷射掃描資料：光達驅動正常發布 /scan 話題（第 8 章的 LaserScan 格式）。
8. 里程計資料：機器人發布 odom 話題與 odom→base_link 的座標變換（第 10.7 節）。這需要輪速編碼器與正確的輪徑、輪距參數。
9. TF 座標樹完整：特別是 base_link→laser 的固定變換（光達安裝位置的外參，第 10 章）。設錯的話，掃描點會與機器人錯位，地圖必定歪掉——這正是第 10 章「地圖歪了先看 TF」口訣的由來。

建圖流程

10. 啟動機器人底層驅動（馬達控制、編碼器里程計、光達）。
11. 確認前置條件：用工具檢查 /scan 有資料、TF 樹完整無斷裂、里程計會隨移動而變化。
12. 啟動 Gmapping 節點，設定關鍵參數（粒子數、地圖解析度、更新距離門檻等）。
13. 開啟 RViz，顯示地圖、雷射掃描、TF 與粒子雲，以便即時觀察建圖狀況。
14. 遙控機器人在環境中緩慢移動，覆蓋所有想建圖的區域。速度要慢——這是成功的關鍵之一。
15. 觀察地圖成長：牆線是否清晰？有無重影？繞迴路後是否閉合？
16. 建圖完成後，用 map_saver 儲存地圖，產生 .pgm 影像檔與 .yaml 描述檔。
17. 日後導航時，以 map_server 載入這張地圖，配合 AMCL（第 3、11 章）進行定位。

關鍵參數

參數	意義	調校建議
粒子數	維持幾個軌跡假設	教室 30 個；環境大或里程計差可增至 50–100
地圖解析度	每格代表的實際邊長	室內常用 0.05 m（第 13.5 節）
更新距離 / 角度門檻	移動多少才更新一次	設太大會漏掉細節，太小則運算浪費
雷射最大可用距離	超過此距離的量測捨棄	略小於光達標稱量程較穩健
里程計雜訊參數	描述里程計有多不準	打滑嚴重的平台應調大

表 14-1　Gmapping 主要參數與調校建議

14.6　常見失敗案例：輪子打滑、場景重複、光達遮蔽

這一節可能是全章對實作最有用的部分。SLAM 失敗是常態而非例外，能辨識失敗模式並對症下藥，是工程能力的核心。

失敗一：輪子打滑與里程計失準

症狀：地圖出現明顯錯位或重影，尤其在轉彎處；機器人「以為」自己走了很遠，地圖卻被拉長。

成因：輪子在光滑地面打滑、地毯與磁磚交界處摩擦力突變、輪徑或輪距參數設定錯誤（這一項最常被忽略，而且會造成系統性的尺度誤差，使整張地圖等比例放大或縮小）。

- 對策：確實校正輪徑與輪距（讓機器人直走 5 公尺、原地轉 10 圈，比對里程計讀值與實測）；放慢速度減少打滑；調大 Gmapping 的里程計雜訊參數，讓系統少信任里程計、多倚賴掃描匹配。

失敗二：場景重複與特徵不足

症狀：長走廊被畫得忽長忽短、牆線重疊；空曠大廳中機器人徹底迷失。這正是第 13.6 節的走廊困境。

成因：沿走廊方向缺乏獨特幾何特徵，掃描匹配無法確定縱向移動量；空曠處則四周都超出光達量程，無從匹配。

- 對策：確保里程計品質良好（此時它是縱向資訊的唯一來源）；增加粒子數以維持更多假設；在走廊沿途放置紙箱等臨時障礙物提供特徵（教學情境常用的技巧）；或改用量程更大的光達。

失敗三：光達遮蔽與看不見的障礙

症狀：地圖上出現不該有的缺口；機器人撞到地圖上沒畫出來的東西。

成因：玻璃與鏡面讓雷射穿透或反射（第 8 章）；黑色吸光物體反射微弱；障礙物不在光達的掃描平面高度上（第 8 章的高度盲區，案例 8-1 的椅腳問題）；或有人站在光達前方形成移動遮蔽。

- 對策：建圖時清空場地、避免人員在機器人周圍走動；玻璃隔間可在建圖期間暫時貼上紙張或放置紙箱；認知到 2D 地圖只反映單一高度切面，導航時需搭配其他感測器補足。

失敗四：移動太快

症狀：掃描匹配跟不上，地圖突然錯位甚至「爆掉」。

成因：機器人移動或轉彎過快，前後兩次掃描差異太大，超出掃描匹配的收斂範圍（第 13.3 節 ICP 需要好的初始猜測）。

- 對策：放慢速度，尤其轉彎時。這是最簡單也最有效的一招——教學經驗中，多數建圖失敗只要「開慢一點」就解決了。

失敗模式	典型症狀	主要對策
輪子打滑	地圖錯位、重影、比例失真	校正輪徑輪距、放慢、調大里程計雜訊
場景重複	走廊變形、機器人迷失	確保里程計品質、增加粒子、加臨時特徵

失敗模式	典型症狀	主要對策
光達遮蔽	地圖有缺口、撞到未畫出的障礙	清場、處理玻璃、認知高度盲區
移動太快	地圖突然錯位或爆掉	放慢速度，尤其轉彎時

表 14-2　Gmapping 常見失敗模式與對策

14.7　教學實作：用小車建立教室地圖

本節說明如何把 Gmapping 落實為一堂可操作的課，提供高中與大學部兩種深度的建議。詳細步驟見本章實作活動。

高中版：先看懂，再動手

高中課程建議以「觀察與體驗」為主，不必深究數學：

- 先用模擬器（TurtleBot3 Gazebo，第 28 章）讓每位同學都能操作，不受硬體數量限制。
- 重點觀察三件事：地圖如何隨機器人移動而「長」出來、粒子雲如何隨移動散開又收斂、以及開太快時地圖如何出錯。
- 刻意示範失敗：讓同學故意把機器人開很快，親眼看到地圖爆掉——失敗的震撼比成功的順利更能建立直覺。
- 連結生活：對照家中掃地機器人 App 的地圖（案例 13-1），理解兩者是同一回事。

大學部版：動手調參，理解機制

大學部應進一步操作實體平台並理解參數影響：

- 親手完成 TF 設定與里程計校正，體會前置條件的重要性（這往往佔掉一半的除錯時間）。
- 系統性地實驗參數：改變粒子數、地圖解析度、里程計雜訊，量化比較地圖品質（用第 13.7 節的評估方法）。
- 在教室、走廊兩種場域各建一次圖，對照第 13.6 節的預測，驗證走廊確實較難。

- 建完地圖後，接續執行 AMCL 定位與導航（第 29 章），完成「建圖→定位→導航」的完整鏈路。

教學心法

依多年教學經驗，建議教師掌握三個要點。其一，先講清楚前置條件與失敗模式，再讓同學動手——否則課堂會淹沒在除錯中。其二，強調「慢」：速度是新手成功率最大的變數。其三，務必安排「評估地圖品質」的環節（第 13.7 節），讓同學養成不只建圖、還要判斷好壞的習慣。這三點能讓一堂 SLAM 實作課的成功率大幅提升。

至此，同學已經完整走過第一個 SLAM 系統。Gmapping 用粒子濾波優雅地解開了雞生蛋問題，但它也有明確的限制：粒子數與地圖相乘的記憶體壓力使它難以應付大場景，且濾波方法無法回頭修正歷史（第 12 章）。下一章的 Cartographer 走另一條路——用第 12 章的圖優化與閉環偵測，讓大範圍建圖與「一鍵拉正」成為可能。兩相對照，同學將更深刻理解濾波與最佳化兩條路線的取捨。

公式或流程

一、Rao-Blackwell 化的機率分解

SLAM 要估計「軌跡與地圖的聯合分布」（第 1 章）。RBPF 的關鍵是把它拆成兩項：

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(m \mid x_{1:t}, z_{1:t}) \cdot p(x_{1:t} \mid z_{1:t}, u_{1:t})$$

右側第二項是「軌跡的分布」——低維，用粒子取樣；右側第一項是「已知軌跡下的地圖分布」——在第 13 章我們已知它可以用對數勝算比直接算出，不需取樣。這條式子就是 14.2 節「拆解」的數學表述，也是 Gmapping 全部巧思的濃縮。

二、粒子權重

第 i 個粒子的權重更新為：

$$w^{(i)} \propto p(z_t \mid m^{(i)}, x_t^{(i)})$$

讀作：權重正比於「以該粒子的地圖 $m^{(i)}$ 與位姿 $x_t^{(i)}$，觀測到 z_t 的可能性」。掃描與地圖越吻合，權重越高。這正是第 11 章觀測模型在 SLAM 中的具體形式。

三、有效粒子數與選擇性重採樣

衡量粒子多樣性的指標：

$$N_{eff} = 1 / \Sigma_i (w^{(i)})^2 \quad （權重已正規化）$$

若所有粒子權重相等（多樣性最佳），N_{eff} 等於粒子總數 N;若權重集中在單一粒子，N_{eff} 趨近 1。Gmapping 的選擇性重採樣規則為：

$$若\ N_{eff} < N / 2，才執行重採樣$$

這條簡單的判斷式，正是 14.4 節避免粒子退化的關鍵機制。

四、Gmapping 單步完整流程

圖 14-4　Gmapping 每一步的處理流程

① 取得新的里程計讀值與雷射掃描

↓

② 對每個粒子：以里程計為初始猜測，執行掃描匹配精修位姿

↓

③ 在精修結果附近的小範圍取樣，得到該粒子的新位姿（改良提議分布）

↓

④ 計算權重：比較「該粒子地圖預期的掃描」與「實際掃描」的吻合度

↓

⑤ 每個粒子用自己的新位姿，更新自己的地圖（第 13 章對數勝算比）

↓

⑥ 計算 N_eff;若低於門檻則重採樣（選擇性重採樣）

↓

⑦ 輸出權重最高粒子的地圖與位姿 → 回到 ①

五、記憶體估算

Gmapping 的記憶體需求為粒子數乘上單張地圖大小：

$$\text{總格子數} = \text{粒子數} \times (\text{寬} / \text{解析度}) \times (\text{高} / \text{解析度})$$

例如 50 個粒子、20 m × 20 m 房間、0.05 m 解析度：50 × 400 × 400 = 800 萬格。這條式子量化了 14.3 節指出的限制——這正是 Gmapping 難以擴展到大場景的根本原因。

14.8　Gmapping 的今天：工具會過時，思想不會

對同學要誠實：如果今天要在 ROS 2 上建一張 2D 地圖，大多數人不會選 Gmapping。它

是 ROS 1 時代的套件，官方生態的主流已經轉向第 13.8 節介紹的 slam_toolbox。那為什麼本書還要花一整章講它？

一、因為 RBPF 並沒有消失

Gmapping 這個軟體套件會退場，但它背後的 Rao-Blackwell 化粒子濾波仍然活躍在三個地方。

第一，第 16 章的 AMCL——掃地機器人與服務機器人在已知地圖中定位，用的就是粒子濾波，只是不必再建地圖。第二，消費級掃地機器人的定位模組大量採用這條路線，因為它在低運算資源下表現穩定、實作簡單。第三，它持續被移植到新的問題上：第 6.7 節介紹的磁力計自動校正就是一個例子——以因式分解的粒子濾波同時估位姿、磁場地圖與感測器零偏，骨架與本章完全相同。

換句話說，本章學的不是一個軟體的使用方法，而是一種解題的結構：當狀態中有一部分「給定其餘部分後可以解析求解」時，就把它因式分解出去，只用粒子處理真正困難的那一部分。這個結構在往後的研究中會反覆派上用場。

二、它的弱點恰好是下一章的主題

多份 2D SLAM 的比較研究得到一致的結論：在走廊、大場域與需要閉環的場景中，以最佳化為後端的方法（Cartographer 一類）在地圖精度上明顯優於 Gmapping。原因在本章已經埋下伏筆——粒子濾波是遞迴的，一旦某一步的所有粒子都偏了，就再也回不來；而重採樣本身又會持續消耗多樣性。

這不是 Gmapping 寫得不好，而是濾波路線的結構性限制，第 11 章的小結已經說過。第 15 章的 Cartographer 正是用第 12 章的位姿圖最佳化補上這一塊：保留全部歷史，閉環時一次修正。兩章連著讀，同學會對「為什麼現代 SLAM 後端幾乎都用最佳化」有切身的理解，而不只是記住結論。

三、粒子濾波自己也在進步

值得一提的是，粒子濾波的研究並未停滯，近年的改良大致有三個方向。其一是提議分布的學習化：用神經網路從資料中學出「該往哪裡撒粒子」，取代本章的掃描匹配。其二是平行化：粒子彼此獨立，天生適合 GPU，這使得「幾千顆粒子」在今天不再是負擔。其三是與最佳化的混合：用粒子處理多峰與不確定性，用最佳化處理局部精修——第 18 章的 LIO-SAM 與第 23 章的 ORB-SLAM3 都可以看到這種分層思路的影子。

給同學的三個判斷

一、學演算法不要只學工具。slam_toolbox 的使用方式幾年就會變，但「因式分解、用粒子表示多峰」這個結構不會。

二、濾波與最佳化不是誰取代誰。第 11.8 節說過：高頻即時用濾波、全域一致用最佳化，一個完整系統兩者並存。

三、本章的實作仍然值得做。即使最後產品不用 Gmapping，親手看著三十顆粒子逐步收斂、看著 N_eff 跌破門檻觸發重採樣，是理解整個機率 SLAM 家族最快的途徑。

案例

案例 14-1　原作者的課堂：從論文到 YouTube

Gmapping 的三位作者 Grisetti、Stachniss 與 Burgard，正是第 2 章介紹的 Freiburg / Bonn 機率機器人學研究群核心成員。有趣的是，Stachniss 教授後來把粒子濾波 SLAM 的完整推導錄成影片免費公開在 YouTube 上，成為全球 SLAM 學習者的共同教材。

這意味著同學讀完本章後，可以直接去看演算法的原創者親自講解——他會用動畫展示粒子如何散開、如何被觀測秤重、如何重採樣收斂。這種「讀教科書理解架構、看原作者影片補足直覺」的組合，是本書第 2 章推薦學習路線的最佳實踐。

這個案例也呼應第 2 章「知識公開的力量」：一個 2007 年發表在頂級期刊的演算法，

今天任何一位台灣的高中生或大學生，都能免費取得原始論文、開源程式碼與作者親授影片。學習 SLAM 的門檻，已從「能否進入名校」變成「有沒有一份好的學習地圖與持續的毅力」——這正是本書想扮演的角色。

案例 14-2　三十個粒子的教室：為何 Gmapping 夠用又不夠用

在大學的教學實驗中，一個典型的觀察是：用 30 個粒子在一般教室建圖，結果相當漂亮——牆線筆直、迴路閉合、細節清晰。但同樣的設定拿到整層樓（數十公尺見方、多個房間與長走廊），地圖品質就明顯下降，甚至失敗。

原因是兩重的。其一是記憶體與運算：場景變大，每張地圖的格子數暴增，乘上粒子數後負擔沉重（公式五）。其二更根本：大場景意味著更長的軌跡、更多的累積漂移，需要更多粒子才能涵蓋可能的假設；但粒子一多，運算又跟不上，形成兩難。

更關鍵的是：當機器人繞完一大圈回到起點，即使閉環被「偵測」到，粒子濾波也無法像圖優化那樣回頭修正整條歷史軌跡——每個粒子的地圖早已依錯誤軌跡畫定，無法重來（第 12 章的核心論點）。這正是第 15 章 Cartographer 要解決的問題。同學若能親手做這個「教室成功、整層樓失敗」的對照實驗，對兩章的差異會有極深刻的體會。

案例 14-3　掃地機器人裡的粒子：理論就在你家

第 11 章案例提過，許多掃地機器人的除錯模式能看到粒子雲收斂的過程。本章讓我們更進一步：掃地機器人的「首次建圖」與「後續清掃」，其實對應兩種不同的技術。

首次進入陌生環境時，機器人必須同時定位與建圖——這是本章的 SLAM。而地圖建好之後，日後每次清掃只需在已知地圖中定位，這就是第 3 章與第 11 章的蒙地卡羅定位（AMCL）。兩者共用粒子濾波的骨架，差別在於「地圖是否已知」：SLAM 的粒子各帶一張成長中的地圖，AMCL 的粒子則共用同一張固定地圖。

理解這個區分很重要，因為它對應 ROS 中兩套不同的工具與流程（14.5 節）：建圖階

段跑 Gmapping、儲存地圖；導航階段用 map_server 載入地圖、跑 AMCL 定位。第 29 章的實作會完整走過這條鏈路。下次看家中掃地機器人第一次清掃時「小心翼翼地探索」、第二次「熟門熟路地直接開工」，你就知道它切換了兩套演算法。

實作活動

實作 14-1　用 Gmapping 建立教室地圖

目的：完整執行一次粒子濾波 SLAM 建圖，觀察粒子雲的演化，並透過參數實驗與失敗重現，理解 Gmapping 的機制與限制。本實作提供模擬與實體兩條路徑。

路徑 A：模擬建圖（全班可做，不需硬體）

12. 啟動 TurtleBot3 Gazebo 模擬環境與 SLAM 節點（第 28 章有完整安裝步驟）。
13. 開啟 RViz，同時顯示：佔據格地圖、雷射掃描、TF 座標軸，以及粒子雲（若工具支援）。
14. 用鍵盤遙控機器人在模擬房間中緩慢移動，一邊移動一邊觀察地圖如何逐格「長」出來、未知的灰色區域如何被填滿。
15. 觀察粒子雲：機器人直線前進時粒子雲如何散開？轉彎或看到明顯特徵時如何收斂？對照第 11 章與 14.3 節的理論。
16. 速度實驗：先用很慢的速度建一次圖，再用最快速度建一次，比較兩張地圖的品質差異，重現 14.6 節「移動太快」的失敗模式。
17. 儲存地圖，用第 13.7 節的方法評估品質：牆是否筆直？迴路是否閉合？量測地圖上的房間尺寸與模擬設定值比較。

路徑 B：實體小車建圖（有硬體的班級）

18. 前置檢查：確認 /scan 有資料、TF 樹完整（特別是 base_link→laser）、里程計會隨移動變化。這一步不可跳過。

19. 里程計校正：讓小車直走 3 公尺，比對里程計讀值與捲尺實測；原地轉 10 圈，比對角度讀值。若誤差大，修正輪徑與輪距參數。

20. 清場：請同學退到教室邊緣，移開可能遮蔽光達的物品，若有玻璃隔間可暫時貼紙。

21. 緩慢遙控小車繞行教室一圈（含四個角落），回到起點，觀察迴路是否閉合。

22. 儲存地圖檔（.pgm 與 .yaml），並在地圖上量測教室尺寸，與捲尺實測值比較，計算誤差百分比。

第二部分：參數實驗（大學部）

23. 粒子數實驗：分別用 5、30、100 個粒子在同一場域建圖，比較地圖品質與運算負擔（可觀察 CPU 使用率與更新頻率）。

24. 解析度實驗：用 0.02、0.05、0.10 m 三種解析度建圖，比較細節呈現與記憶體用量，驗證公式五的估算。

25. 走廊挑戰：把機器人開進長走廊建圖，對照第 13.6 節與 14.6 節的預測，觀察是否出現變形。嘗試在走廊沿途放置紙箱提供特徵，再建一次，比較改善程度。

26. 打滑實驗：在地面鋪一塊光滑板材或讓一側輪子短暫懸空，製造打滑，觀察地圖如何出錯。

討論問題

5. 快速移動與緩慢移動建出的地圖差多少？從 14.6 節與第 13.3 節的角度，解釋為什麼「開慢一點」如此有效。

6. 粒子數從 5 增加到 100，地圖品質改善了多少？運算負擔增加多少？你認為在你的場域，最合適的粒子數是多少？

7. 走廊建圖出現了什麼問題？放置紙箱後改善了嗎？這印證了 14.6 節的哪一項論述？

8. 若機器人繞了一大圈回到起點但地圖沒有閉合，Gmapping 有辦法「事後修正」整張地圖嗎？為什麼？（提示：第 12 章與案例 14-2）

延伸挑戰（大學部）

其一：用 Python 實作一個極簡的 RBPF 模擬——在程式中定義一個虛擬房間，建立 20 個粒子（各自有位姿與一張小地圖），實作「移動加雜訊→秤重→選擇性重採樣→各自更新地圖」的循環，把每個粒子的地圖並排畫出，觀察正確的假設如何勝出。其二：建完地圖後，接續啟動 map_server 與 AMCL，讓機器人在自己建的地圖中定位並執行導航（第 29 章），完成「建圖→定位→導航」的完整鏈路。

延伸閱讀

8. G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," IEEE Transactions on Robotics, 2007.（Gmapping 的原始論文，本章主要依據）
9. M. Montemerlo et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI, 2002.（RBPF-SLAM 的開創性工作，與 Gmapping 互為對照）
10. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（第 13 章 FastSLAM 與粒子濾波 SLAM 的完整理論）
11. Cyrill Stachniss, Particle Filter 與 FastSLAM 系列課程影片（YouTube）.（原作者親自講解，搭配本章閱讀效果最佳）
12. OpenSLAM Gmapping 開源專案與 ROS slam_gmapping 套件文件.（原始程式碼與參數說明）
13. slam_toolbox 開源專案（GitHub）.（ROS 2 環境中基於圖優化的現代 2D SLAM，可與 Gmapping 對照實驗）
14. ROBOTIS TurtleBot3 e-Manual 之 SLAM 章節.（模擬與實體建圖的完整操作指引）

從粒子到圖：本章的位置回顧

本章讓同學看到了 SLAM 第一次真正運作起來的樣子：位姿與地圖同時被估計，而且估得還不錯。這是第四篇的第一個完整系統。

但圖 14-3 左圖那條曲線也誠實地說明了它的天花板。即使把粒子數加到一百，平均誤差也只降到 0.08 公尺，而且運算量已經是三十顆的三倍以上。更根本的問題是：粒子濾波的所有修正都發生在「當下」——它沒有任何機制可以回頭改寫走過的軌跡。

想像機器人繞了整棟樓一圈回到起點，發現眼前的景象與地圖上記錄的起點對不上。Gmapping 能做的只有從這一刻起慢慢修正，中間那一整圈已經寫進每顆粒子地圖裡的誤差，再也拿不出來。這就是第 13.7 節所說的「漂移」病徵，而它的解藥是閉環。

第 15 章的 Cartographer 用三件事解決這個問題：把掃描累積成子圖（Submap）以降低位姿圖的規模、用分支定界法在全圖中高效搜尋閉環候選、再用第 12 章的位姿圖最佳化一次修正所有子圖的位置。同一個教室、同一台小車，第 15 章會讓同學親眼比較兩者建出的地圖差在哪裡——那時再回頭看本章，Gmapping 的優點與極限都會變得非常具體。

小結

8. Gmapping 由 Grisetti、Stachniss 與 Burgard 於 2007 年提出，是最著名的 2D 雷射 SLAM 系統之一；輸入為光達掃描、里程計與 TF，輸出為佔據格地圖與 map→odom 變換。
9. Rao-Blackwell 化把 SLAM 拆成兩部分：軌跡（低維，用粒子取樣）與地圖（已知軌跡即可解析求出）；每個粒子是一個「平行世界」，各帶一條軌跡與一張地圖，以觀測評分競爭，漂亮地解開雞生蛋問題。
10. 粒子數是精度與運算的權衡（教室約 30 個）；權重代表假設與觀測的吻合度；重採樣讓好假設存活，但過度重採樣會造成粒子退化，使錯誤假設獨佔而無法挽回。

11. Gmapping 的兩項關鍵改良：以掃描匹配改良提議分布（讓粒子撒在合理位置，所需粒子數從數千降至數十）、以及選擇性重採樣（只在有效粒子數低於門檻時才重採樣，避免退化）。
12. ROS 實作的三個前置條件缺一不可：/scan 雷射資料、里程計與 odom→base_link 變換、完整正確的 TF 樹（尤其 base_link→laser 外參）；建圖後以 map_saver 存成 .pgm 與 .yaml。
13. 四大失敗模式：輪子打滑（校正輪徑輪距、調大里程計雜訊）、場景重複（走廊，靠里程計與增加特徵）、光達遮蔽（玻璃、黑色物體、高度盲區）、移動太快（放慢是最有效的一招）。
14. Gmapping 的根本限制在於記憶體隨「粒子數×地圖大小」成長，難以應付大場景；且身為濾波方法，無法回頭修正歷史軌跡與地圖——這正是下一章圖優化路線要解決的問題。

本章完成了第一個完整的 SLAM 系統。Gmapping 以粒子濾波優雅地解開雞生蛋問題，但也讓我們清楚看到濾波路線的天花板：大場景吃不消、歷史無法修正。下一章 Cartographer 登場——它由 Google 開發，以子地圖（Submap）組織空間、以第 12 章的圖優化執行閉環修正，能在大範圍場景中即時建圖，並在偵測到閉環的瞬間把整張歪掉的地圖「一鍵拉正」。第 12 章那個令人震撼的畫面，即將在真實系統中重現。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 15 章

Cartographer：Google 2D / 3D SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

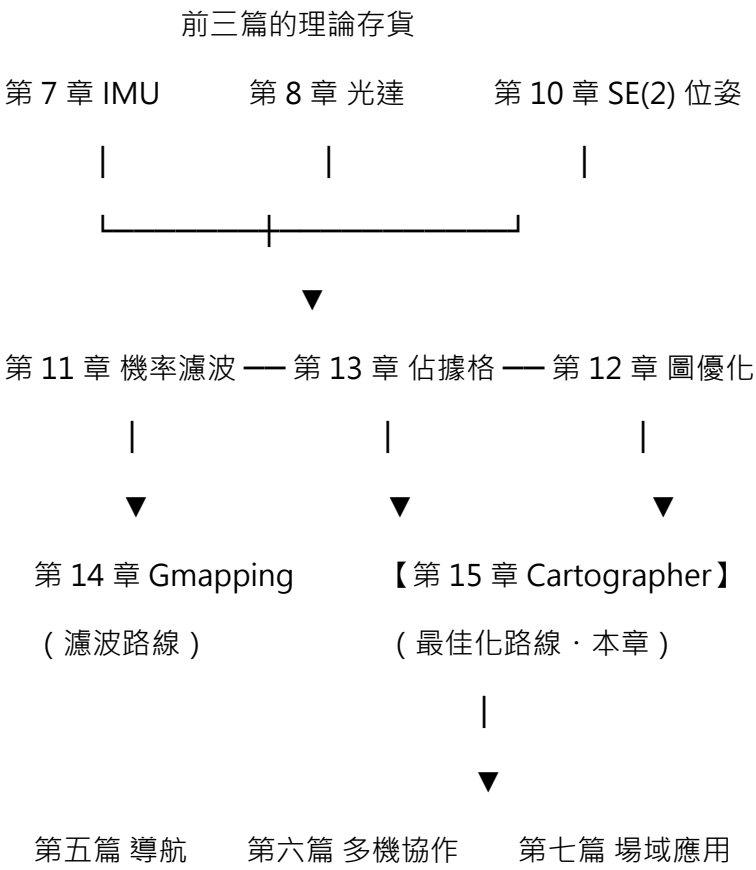
大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第四篇走到這裡，兩條理論主線終於各自長出了一套完整的系統。第 11 章的貝氏濾波，在第 14 章長成了 Gmapping；第 12 章的圖優化，就在本章長成 Cartographer。兩章不是並列的替代品，而是同一個問題在兩種數學框架下的答案，也因此，把它們放在一起讀，比分開讀各自多出一倍的收穫。

往回看，本章幾乎動用了前三篇的全部存貨：第 8 章的光達規格決定了掃描匹配的素材品質，第 7 章的 IMU 在 3D 模式中幾乎是必要條件，第 10 章的 SE(2) 位姿合成是子地圖拼貼的底層語言，第 11 章的機率格概念被第 13 章具體化成子地圖的內部表示，而第 12 章的稀疏位姿調整，就是本章全域 SLAM 的引擎本身。往前看，本章產出的 .pbstream 地圖與閉環修正能力，會在第五篇的導航、第六篇的多機與第七篇的場域應用裡一再被取用。

圖 15-0　第 15 章的連結地圖



學習目標

上一章的 Gmapping 以粒子濾波解開了雞生蛋問題，但也暴露了濾波路線的天花板：大場景吃不消、歷史無法修正。本章介紹 Google 開發的 Cartographer——它以子地圖組織空間、以第 12 章的圖優化執行閉環修正，能在大範圍場景中即時建圖，並在偵測到閉環的瞬間把整張歪掉的地圖「一鍵拉正」。學完本章之後，同學應能夠達成下列目標：

14. 說明 Cartographer 的雙子系統架構：局部 SLAM 與全域 SLAM 的分工。
15. 解釋子地圖（Submap）的概念，以及為何要把掃描分組成子地圖。
16. 說明局部與全域兩種掃描匹配的目的與差異。
17. 說明分支定界法如何讓即時閉環偵測成為可能。
18. 說明在 ROS 中執行 2D Cartographer 建圖的流程與關鍵設定。
19. 說明 3D Cartographer 與背包式空間建模的應用。
20. 比較 Cartographer 與 Gmapping 的架構、優缺點與適用場合。

核心概念

中文名詞	英文名詞	概念說明
子地圖	Submap	由數十筆掃描組成的小塊機率格地圖，是 Cartographer 的基本單元。
局部 SLAM	Local SLAM	前端：把新掃描插入當前子地圖，局部一致但會漂移。
全域 SLAM	Global SLAM	後端：在背景尋找閉環約束並最佳化整個位姿圖。
機率格	Probability Grid	子地圖的內部表示，每格記錄佔據機率。
掃描匹配	Scan Matching	把新掃描對齊到子地圖或既有子地圖的過程。
閉環約束	Loop Closure Constraint	連接重訪位置的邊，用來消除累積誤差。

中文名詞	英文名詞	概念說明
分支定界	Branch and Bound	有效率地搜尋最佳匹配、剔除不可能解的演算法。
多解析度預算格	Precomputed Grids	事先算好的不同解析度地圖，加速分支定界搜尋。
稀疏位姿調整	Sparse Pose Adjustment	對子地圖與掃描位姿構成的圖執行的最佳化。
非線性最小平方	Nonlinear Least Squares	Cartographer 以 Ceres 求解的最佳化形式。
背包式建模	Backpack Mapping	人員背負感測器組行走以掃描建築空間。
穩健核函數	Robust Kernel	降低錯誤閉環影響的最佳化技巧。

15.1 Cartographer 系統架構

Cartographer 由 Google 開發並於 2016 年開源，其核心論文標題直接點出了它的主打特色：「2D 光達 SLAM 中的即時閉環」。這個名稱本身就宣示了它與 Gmapping 的根本差異——它把閉環偵測與圖優化做進了系統核心，而且能即時執行。

兩個子系統：局部與全域

理解 Cartographer，最重要的是掌握它的雙子系統架構。它可以被視為兩個獨立但相關的子系統：局部 SLAM（也稱為前端或局部軌跡建構器）負責建立一連串的子地圖，每個子地圖在局部上是一致的，但我們接受局部 SLAM 會隨時間漂移；全域 SLAM 則在獨立的執行緒中運行，尋找子地圖之間的閉環約束。

- 局部 SLAM（前端）：把每一筆新掃描匹配並插入「當前子地圖」。它追求即時與局部精確，但明知會隨時間累積漂移——這是刻意的設計取捨。
- 全域 SLAM（後端）：在背景執行緒中，把掃描與附近的子地圖做匹配以建立閉環約束，並持續最佳化由子地圖與掃描位姿構成的約束圖。

關鍵在於：兩者同時運行。前景快速建圖讓操作者立即看到結果，背景默默尋找閉環並修正全域一致性。這正是它能「即時」的原因，也是與 Gmapping 最大的架構差異。

圖 15-1 Cartographer 雙子系統架構

【局部 SLAM（前端）——前景執行】

新掃描 → 掃描匹配到當前子地圖 → 插入子地圖 → 子地圖完成後封存

特性：即時、局部一致、會漂移

【全域 SLAM（後端）——背景執行緒】

掃描 ↔ 附近既有子地圖做匹配 → 找到好匹配則建立閉環約束

↓

把子地圖位姿與掃描位姿組成位姿圖（第 12 章）

↓

每隔數秒以 Ceres 求解非線性最小平方 → 整張地圖被拉正

兩者同時運行：前景給即時預覽，背景保證全域一致

與第 12 章的連結

同學應該立刻認出來：全域 SLAM 做的正是第 12 章的圖優化。它結合了局部掃描匹配形成的子地圖（以 Ceres 為基礎的非線性最小平方最佳化器精修的機率格），與帶有分支定界閉環偵測的稀疏位姿圖，在一般等級的硬體上達成精確的即時 2D SLAM。第 12 章提到 Cartographer 使用 Ceres 函式庫，在此得到印證。理論與系統，在這一章正式接軌。

15.2 Submap 子地圖

子地圖是 Cartographer 最具識別度的設計。理解它為何存在，就理解了 Cartographer 為何能處理大場景。

什麼是子地圖

子地圖是一小塊機率格地圖（第 13 章的佔據格），由連續數十筆雷射掃描累積而成。機器人移動時，新掃描不斷插入當前子地圖；當累積到設定的掃描數，這個子地圖就「完成」並被封存，系統改建下一個子地圖。整個環境因此被切成一系列部分重疊的小塊地圖。

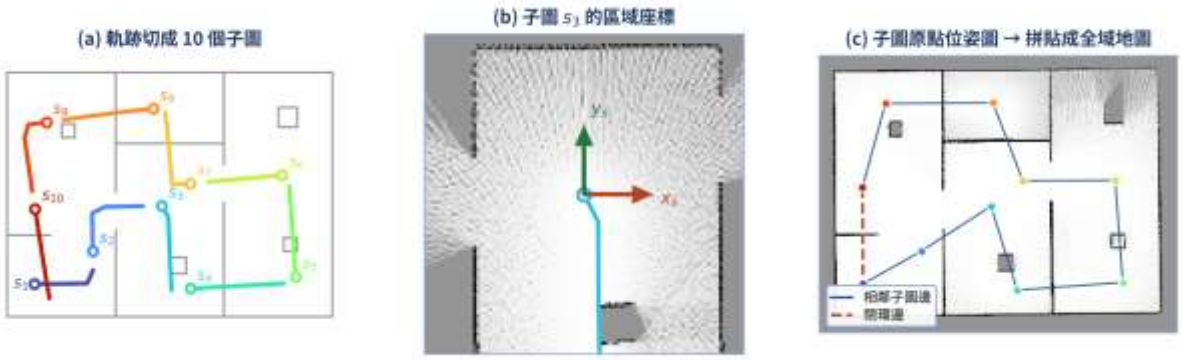
為什麼要分子地圖：三個理由

18. 限制誤差累積：由於掃描只與包含少數近期掃描的子地圖做匹配，這個做法會緩慢累積誤差；對於僅僅數十筆連續掃描而言，累積的誤差很小。換句話說，在一個子地圖的短暫生命週期內，漂移小到可以忽略——每個子地圖內部是「乾淨」的。

19. 提供最佳化的單元：全域最佳化時，不需要調整每一筆掃描，只需調整「子地圖的位姿」。一個子地圖代表數十筆掃描，這讓位姿圖的節點數大幅減少，最佳化因此能即時完成。這是效率的關鍵。

20. 成為閉環比對的對象：閉環偵測時，是拿「當前掃描」去比對「過去的子地圖」。子地圖比單筆掃描包含更多資訊、更穩定，比對結果也更可靠。

圖 15-2　子圖的三個面向：分段、區域座標、拼貼



這張圖把「子地圖」這個抽象名詞拆成三個可以用眼睛確認的面向。左圖是同一條軌跡被切成十段，每一段就是一個子地圖 s_1 至 s_{10}；空心圓標出的是每個子地圖的原點位姿，也就是這段路程開始時機器人所在的位置與朝向。請注意：切分的依據不是走了多遠，而是累積了多少筆掃描——Cartographer 以掃描數為單位封存子地圖，正是為了讓每個子地圖內部的

誤差被控制在可忽略的量級。

中圖把第三個子地圖單獨拿出來，放進它自己的區域座標系。紅色與綠色箭頭就是這個子地圖的 x 軸與 y 軸，原點永遠落在這段路程的起點。子地圖內部完全不知道自己在世界的哪裡，它只記得「相對於我的原點，這面牆在哪、那個轉角在哪」。這個刻意的失憶，正是 Cartographer 能把大場景切小的關鍵：區域的歸區域，全域的歸全域。

右圖是最後的拼貼結果。藍線連起相鄰子地圖的原點，構成一條位姿鏈；紅色虛線則是閉環邊，把最後一個子地圖拉回第一個。真正被最佳化的變數，只有圖上那十個彩色圓點——十個子地圖原點的位姿，合計三十個未知數。相對之下，這段軌跡本身有近六十個時刻；若逐一最佳化每一筆掃描的位姿，未知數會多出一倍，而最佳化的計算量大致隨未知數成長。這就是第 15.2 節所說「提供最佳化的單元」的具體意思。

把三張圖連起來看，子地圖的角色就清楚了：它既是誤差的隔離艙（區域誤差不外溢），也是最佳化的樂高積木（一塊代表數十筆掃描），還是閉環比對的對象（資訊量足夠，比單筆掃描穩定）。一個設計同時解掉三個問題，這是 Cartographer 最值得學習的工程品味。

子地圖要切多大？

太小：子地圖數量暴增，位姿圖節點變多，全域最佳化變慢；而且每個子地圖包含的幾何特徵太少，閉環比對容易誤判。

太大：子地圖內部累積的掃描匹配誤差開始顯著，「區域誤差可忽略」的前提被破壞，閉環拉正後牆線仍然歪斜。

實務起點：2D 室內場域常用 70～100 筆掃描封存一個子地圖（表 15-1 的 num_range_data）；走廊等特徵稀少處宜調小，雜亂房間可調大。

判斷方法：在 RViz 中看子地圖之間的重疊區。相鄰子地圖的牆線若能重合，大小就設對了。

與 Gmapping 的對比

這裡可以清楚看出兩套系統的哲學差異。Gmapping 讓每個粒子維護一整張完整地圖（第 14 章），記憶體隨粒子數乘上地圖大小成長；Cartographer 則只維護一張由眾多子地圖拼成的地圖，靠調整子地圖位姿來修正全域一致性。

這個差異的後果很直接：Cartographer 的記憶體需求與場景大小成正比（而非乘上粒子數），因此能處理整棟建築；而且因為保留了子地圖與位姿圖，它能「回頭修正」——這正是第 12 章指出的圖優化相對濾波的核心優勢。

15.3 Scan Matching

Cartographer 中有兩種掃描匹配，目的截然不同。分清楚它們，是理解系統運作的關鍵。

一、局部掃描匹配：插入當前子地圖

這是前端的日常工作。每來一筆新掃描，系統要決定「它該貼在當前子地圖的哪個位置」。做法是以里程計或 IMU 提供初始猜測（第 13.4 節的思想），再用最佳化方法精修，使掃描點與子地圖的機率格盡可能吻合。

Cartographer 把這個匹配表述為非線性最小平方問題，以 Ceres 求解（第 12 章）。這個做法與傳統 ICP（第 13.3 節）不同——它不是找點對點的對應，而是直接最大化「掃描點落在高佔據機率格」的程度，這對雜訊較不敏感。

二、全域掃描匹配：尋找閉環

這是後端的工作，難度高得多。問題是：當前掃描，是否與某個很久以前建立的子地圖吻合？若吻合，就找到了閉環。

困難之處在於搜尋空間。閉環候選可能在任何位置、任何角度，而且由於已經累積了漂移，初始猜測可能偏差很大——不能只在小範圍內找。若用暴力搜尋，把每個可能的位置

與角度都試一遍，運算量將大到無法即時執行。

這正是 Cartographer 最精妙的技術貢獻要解決的問題，也是下一節的主題。

限制搜尋範圍的實務考量

即使有高效演算法，系統仍需控制計算量。為了限制約束（與計算量）的數量，Cartographer 只考慮所有鄰近節點的一個子取樣集合來建立約束，這由一個取樣比例常數控制；取樣太少可能導致錯過約束、閉環失效，取樣太多則會拖慢全域 SLAM，使即時閉環無法達成。這是實作調參時的重要權衡。

15.4 閉環偵測與全域最佳化

本節是全章的技術核心：Cartographer 如何做到「即時」閉環——這正是它論文標題所宣稱的成就。

分支定界：剪掉不可能的分支

面對龐大的搜尋空間，Cartographer 採用分支定界法（Branch and Bound）。它以分支定界機制在不同的格地圖解析度上運作，有效率地剔除不正確的匹配。

分支定界的核心思想，可以用一個生活比喻理解：假設你要在一整棟大樓裡找一個人。笨方法是逐間房間搜遍；聰明的做法是先在每一層樓快速掃視（低解析度），若某一層明顯不可能有人（例如整層在裝修），就跳過整層不必逐間找——這就是「剪枝」。

Cartographer 的做法正是如此：它把既有子地圖降尺度成幾種不同解析度（例如四分之一、二分之一大小）作為掃描匹配的輸入，匹配過程從低解析度開始做初步的低精度計算，得到最佳分數的初步結果後，用它跳過高解析度中大量不可能的平移旋轉組合，如此反覆直到抵達完整解析度。系統會預先算好尺寸為 1、4、16、64 的多解析度格網來支援這個搜尋。

為什麼這樣就能即時

關鍵在於「上界」的概念。在低解析度地圖上算出的分數，是該區域在高解析度下所有

可能匹配分數的上界（不會低估）。因此若某區域在低解析度下的分數，已經低於目前找到的最佳答案，那麼它在高解析度下也絕不可能更好——整個區域可以安全地跳過，不必展開。

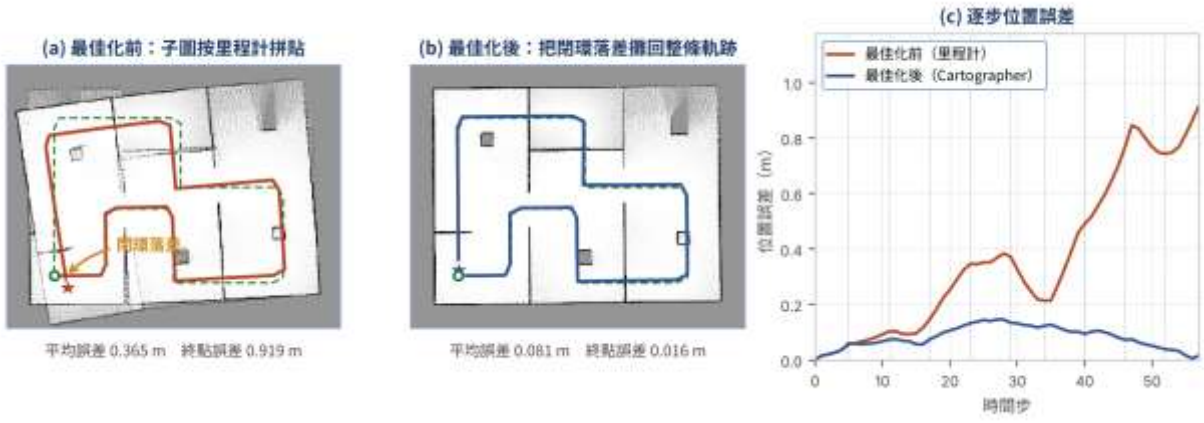
這讓搜尋量從「所有可能組合」降到「少數有希望的分支」，即時閉環因此成為可能。這個機制在 Cartographer 論文中有詳盡的說明，而其專用的快速相關掃描匹配器正是為了讓即時閉環的掃描匹配成為可能而設計的。

全域最佳化：把矛盾攤平

找到閉環約束後，就進入第 12 章的圖優化。閉環最佳化與掃描匹配一樣，也被表述為非線性最小平方問題，這讓系統能輕易加入殘差以納入額外資料；每隔幾秒，系統就用 Ceres 計算一次解。

這個最佳化調整的是「子地圖位姿」與「掃描位姿」，稱為稀疏位姿調整（Sparse Pose Adjustment）。當閉環約束與里程計約束矛盾時，最佳化把矛盾均攤到整條軌跡上——第 12 章那個「整張歪掉的地圖唰地被拉正」的畫面，在此真實上演。

圖 15-3　閉環前後：一條約束如何把整張地圖拉正



這是本章最值得停下來看的一張圖，因為它把案例 15-1 所說「課堂上最震撼的三秒鐘」變成了可以量化的數字。圖(a)是最佳化之前：子地圖純粹按里程計的推算拼在一起。整張地圖明顯歪斜，外牆沿著對角線整片旋轉，而機器人繞完一圈後的終點（星號）並沒有回到出

發點（圓圈），兩者差了 0.92 公尺。金色箭頭指的就是這個落差——它不是感測器的錯，而是數十次微小角度誤差一路累積的結果。

圖(b)是加入一條閉環邊、重新求解位姿圖之後。請注意變化的性質：不是把終點硬生生搬回起點，而是把這 0.92 公尺的矛盾，依照各段約束的可信度攤回整條軌跡。結果是整張地圖轉正、牆線收斂成單一線條，終點誤差降到 0.016 公尺，平均誤差從 0.365 降到 0.081 公尺——降幅接近八成。這正是第 12.3 節「共變異數決定誰讓步」在真實數據上的樣子。

圖(c)把兩條軌跡的逐步誤差畫在一起，是整張圖最有教育意義的一格。紅線（最佳化前）呈現典型的漂移形狀：前段幾乎貼著零，中段緩升，後段急遽翹起——誤差不是線性成長，而是隨著角度偏差被後續路程放大。藍線（最佳化後）則被壓成一個平緩的拱形：最高點出現在軌跡中段（約 0.15 公尺），兩端則被起點與閉環邊同時錨住而趨近於零。

這個拱形值得同學記住，因為它是位姿圖最佳化的指紋。當一條軌跡的兩端都有強約束、中間只靠里程計串接時，誤差的分布必然是兩端低、中間高。它同時也告訴我們一件實務上的事：如果場域能走出兩個以上的迴路，就會有兩條以上的閉環邊，拱形會被進一步壓低。這就是第 15.5 節建圖流程要求「務必走完整個迴路」的理由，也是下一節比較實驗的伏筆。

閉環拉正與「地圖突然跳一下」

為什麼是跳的：全域最佳化每隔數秒執行一次，一旦找到閉環約束，整組子地圖位姿同時被更新，畫面上就是整張地圖瞬間重排——這就是案例 15-1 的三秒鐘。

為什麼導航要小心：地圖與 map→odom 變換同時改變，正在執行的路徑可能瞬間失效。第五篇的導航堆疊必須能承受這種不連續，這也是實務上常把「建圖」與「導航」分成兩個階段的原因。

怎麼確認拉正成功：看迴路接縫處。拉正前牆會有重影或錯開（第 13.7 節的病徵），拉正後應收斂成單一條線。

沒拉正怎麼辦：多半是閉環分數門檻過高或取樣比例過低（表 15-1），先放寬門檻

再試；但門檻放太寬會引來錯誤閉環，請搭配第 15.4 節的穩健核函數一起調。

穩健化：防止錯誤閉環的災難

閉環偵測有可能誤判（例如兩個長得很像的房間）。一條錯誤的閉環約束，會把地圖硬拉到錯誤的位置，造成災難性後果（第 12 章實作活動提過）。因此 Cartographer 在最佳化中採用穩健核函數，降低離群約束的權重，讓少數錯誤匹配不至於毀掉整張地圖。這呼應第 12 章延伸挑戰中「加入錯誤閉環邊」的實驗——現在同學知道實務系統是怎麼防範的了。

15.5　2D Cartographer 實作

本節說明在 ROS 中執行 2D Cartographer 建圖的實務要點。與第 14.5 節的 Gmapping 流程對照閱讀，能看出兩者的異同。

前置條件

與 Gmapping 相同的三要件仍然適用：雷射掃描（/scan）、里程計、完整的 TF 座標樹（第 10 章）。但 Cartographer 有兩點不同：

- IMU 的角色更重要：Cartographer 能有效利用 IMU 提供的姿態資訊（第 7 章），尤其在 3D 模式中幾乎是必要的。2D 模式若有 IMU，建圖穩定度會明顯提升。
- 里程計並非絕對必要：由於局部掃描匹配的能力較強，Cartographer 在特徵豐富的環境中即使沒有里程計也能運作。但在走廊等特徵稀少處（第 13.6 節），里程計仍然關鍵。

設定檔：Lua 組態

Cartographer 的一大特色（也是初學者的門檻）是它使用 Lua 語言的設定檔，參數數量遠多於 Gmapping。大部分局部 SLAM 的選項可以在 2D 的 trajectory_builder_2d.lua 與 3D 的 trajectory_builder_3d.lua 設定檔中找到。

關鍵參數群組包含：

參數群組	控制什麼	調校方向
子地圖掃描數	每個子地圖累積幾筆掃描	太少則子地圖不穩、太多則漂移進子地圖內
解析度	機率格的格子邊長	室內常用 0.05 m（第 13.5 節）
局部匹配權重	掃描匹配時各項殘差的相對重要性	里程計可靠則提高其權重
閉環取樣比例	考慮多少節點來建立約束	太低錯過閉環、太高拖慢即時性（15.3 節）
閉環分數門檻	匹配分數多高才接受為閉環	太低易誤判、太高則閉環偵測不到
最佳化週期	多久執行一次全域最佳化	通常數秒一次

表 15-1　2D Cartographer 主要參數群組

建圖流程

21. 確認感測器資料與 TF 樹正常（同第 14.5 節）。
22. 準備 Lua 設定檔，依平台調整感測器組態（有無 IMU、有無里程計、光達數量）。
23. 啟動 Cartographer 節點與 RViz，顯示子地圖、軌跡與雷射掃描。
24. 緩慢遙控機器人建圖。特別留意：走完迴路回到起點時，觀察是否觸發閉環——這是本章最值得觀察的一刻。
25. 建圖完成後，執行「結束軌跡」與最終最佳化，再匯出地圖檔（.pbstream 格式，可再轉為 ROS 標準的 .pgm 與 .yaml）。

觀察重點：子地圖與閉環

在 RViz 中，Cartographer 會把各個子地圖以不同方式顯示，同學可以清楚看到地圖是由一塊塊子地圖拼成的。而最精彩的時刻是閉環觸發的瞬間：原本錯開的軌跡與地圖會突然「跳」到正確位置。務必安排課堂時間讓同學親眼見證這一刻——這是第 12 章抽象理論最

有力的具象化。

15.6 3D Cartographer 與背包式空間建模

Cartographer 不只做 2D。它同樣支援 3D SLAM，而 Google 開發它的原始動機之一，正是室內空間的三維測繪。

3D 模式的差異

- 感測器：需要 3D 光達（多線，第 8 章）。Google 的系統最初搭載兩具 Hokuyo 雷射掃描儀與一個慣性量測單元，後來的版本則改用兩具 Velodyne PUCK 掃描儀。
- IMU 的必要性：3D 模式下，姿態（滾轉與俯仰，第 7 章）的估計至關重要，IMU 幾乎是必備的。
- 子地圖為 3D：機率格從平面變成立體的體素（Voxel）格，記憶體需求大幅上升。
- 運算負擔：3D 掃描匹配與最佳化的計算量遠高於 2D，對硬體要求較高。

背包式空間建模

第 8 章案例 8-3 提過背包式移動掃描，Cartographer 正是這類系統的代表。操作員背負搭載光達與 IMU 的背包系統，一邊行走一邊即時建立整棟建築的三維模型。

這個應用之所以重要，是因為它解決了傳統測繪的痛點：傳統靜態雷射掃描儀必須在每個定點架設、掃描、再搬到下一點，耗時費力；背包式系統則「走到哪掃到哪」，效率提升數十倍。

- 應用場景：建築資訊模型（BIM）建置與驗證、古蹟與文化資產數位保存、廠房設施管理、室內導航地圖製作、災後結構評估。
- 對大學場域的意義：這是極具潛力的專題方向——用 3D SLAM 掃描校園建築、實驗室或工廠，產生可用於教學與研究的數位分身，把第五篇（第 17、18 章）的 3D 點雲技術實際應用出來。

連續時間 SLAM 的延伸研究

值得一提的是，學術界持續在 Cartographer 的基礎上改良。例如有研究團隊指出，把掃描視為「連續時間軌跡」而非離散時刻的快照，能進一步改善 3D 建圖品質——這類研究正是建立在對 Cartographer 架構的深入理解之上。對有志研究的同學而言，理解本章內容，就是進入這個領域的門票。

15.7 Cartographer 與 Gmapping 比較

本篇的兩大主角至此都已介紹完畢。本節做系統性的對照，這也是第四篇最重要的一張表。

比較項目	Gmapping（第 14 章）	Cartographer（本章）
理論基礎	粒子濾波（第 11 章）	圖優化（第 12 章）
地圖組織	每個粒子一張完整地圖	一系列子地圖拼接
閉環處理	無專門機制，難以事後修正	核心功能，可即時拉正全圖
記憶體	粒子數 × 地圖大小	與場景大小成正比
大場景	受限	擅長，可建整棟建築
3D 支援	無（僅 2D）	支援 2D 與 3D
IMU 利用	不使用	可有效整合，3D 幾乎必備
參數複雜度	較少，易上手	較多（Lua 設定檔），門檻較高
教學價值	理解粒子濾波的最佳範例	理解圖優化與閉環的最佳範例

表 15-2　Cartographer 與 Gmapping 的系統比較

圖 15-4 同一場域、同一條路線：兩套系統的正面對決

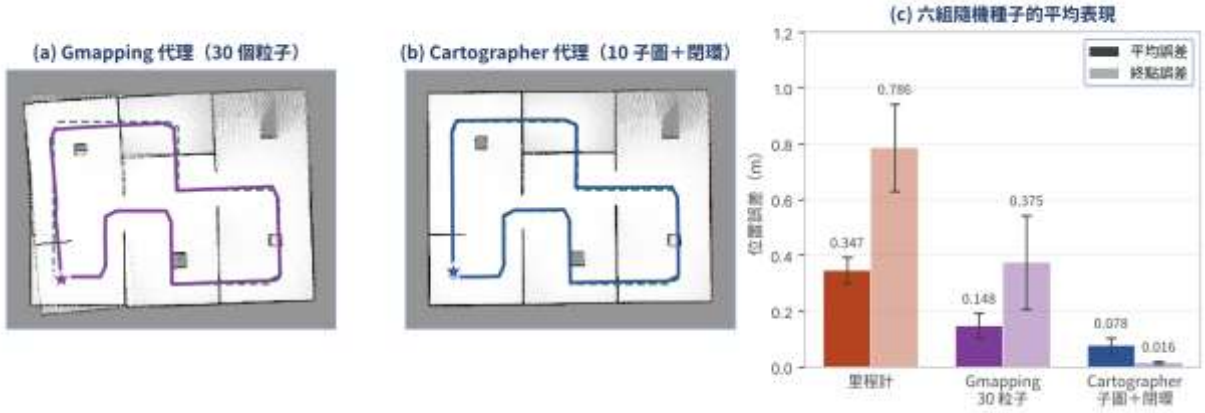


表 15-2 列出了兩套系統的架構差異，這張圖則把差異換算成數字。實驗條件完全一致：同一個 12×9 公尺的 L 形室內場域、同一條繞行一圈回到起點的路線、同一組里程計雜訊參數。圖(a)是第 14 章的 Gmapping 代理（30 個粒子），圖(b)是本章的 Cartographer 代理（10 個子地圖 + 一條閉環邊），圖(c)則是六組不同隨機種子下的統計結果。

先看地圖。兩張都比純里程計的結果好得多，但差別在接縫：Gmapping 的地圖在迴路接合處仍有輕微錯位，外牆帶著一點整體傾斜；Cartographer 的地圖則牆線筆直、迴路乾淨閉合。原因在圖(c)的數字裡——Gmapping 的終點誤差平均 0.375 公尺、標準差 0.168，Cartographer 則是 0.016 公尺、標準差僅 0.004。兩者差了超過二十倍，而且後者幾乎不隨種子波動。

平均誤差的對比同樣明確：里程計 0.347、Gmapping 0.148、Cartographer 0.078 公尺，也就是說粒子濾波把漂移砍掉約六成，而子地圖加閉環又在此之上再砍掉近一半。但更值得注意的是誤差棒的長度。Gmapping 的終點誤差標準差高達 0.168 公尺，接近它自己的平均值一半，這正是粒子濾波的隨機性：同樣的參數，有時運氣好、有時粒子耗盡。Cartographer 的誤差棒則短到幾乎看不見。

這個差異背後有一個結構性的理由，值得同學想清楚：Gmapping 是濾波器，它只能向前走，已經重採樣掉的歷史無法回頭修正，所以終點誤差一旦累積就只能帶著走；Cartographer 保留了整張位姿圖，閉環一旦出現，過去所有子地圖的位姿都可以被重新調

整。「能不能修改歷史」，就是濾波與最佳化兩條路線最根本的分野，也是表 15-2 中所有條目的共同源頭。

> **關於這組對照實驗的誠實說明**
>
> 這是教學用的代理實作，不是 Gmapping 與 Cartographer 原始碼的效能評比。兩者都以本書的模擬場域重寫，以凸顯架構差異，實際套件的表現受參數、感測器與場域影響甚大。
>
> 結果並非在所有條件下都成立。若場域走不出迴路（例如一條直走到底的走廊），Cartographer 沒有閉環可用，優勢會大幅縮小——這正是討論問題第三題要同學思考的。
>
> 粒子數與子地圖數都會改變結果。本圖固定 30 粒子與 10 子地圖，同學可在實作 15-1 中自行掃描這兩個參數。
>
> 請同學用實作 15-1 第二部分在真實套件上重跑一次，並把你量到的數字與本圖比較。數字不同是正常的，重要的是趨勢是否一致。

如何選擇

- 小場域教學、理解粒子濾波：Gmapping。它參數少、概念直觀，是入門的好選擇。
- 大場域、需要閉環、要建整棟樓：Cartographer。它的架構就是為此而生。
- 需要 3D 建模、背包式掃描：Cartographer 3D。
- ROS 2 環境的現代 2D SLAM：除了 Cartographer，也可考慮 slam_toolbox——它同樣基於圖優化，參數較簡潔，近年在 ROS 2 社群中相當流行。

兩章的教學意義

把第 14 與 15 章並列閱讀，同學會發現：它們正是第 11 章（濾波）與第 12 章（最佳化）兩條理論路線的實體化身。Gmapping 展示了粒子濾波的巧思與天花板，Cartographer 展示了圖優化如何突破那個天花板。這不只是兩個工具的比較，而是 SLAM

十餘年技術演進的縮影。

至此，同學已經掌握 2D SLAM 的兩大系統。下一章我們把鏡頭轉向最貼近生活的應用——掃地機器人。從隨機碰撞到 SLAM 規劃、從房間分割到覆蓋路徑規劃，看這些理論如何在你家客廳裡運作，並理解家庭場景中桌椅、鏡面與低矮障礙物帶來的真實挑戰。

15.8 Cartographer 之後：2024–2026 年的 2D SLAM 新局

Cartographer 的論文發表於 2016 年，以本書寫作的時點來看，已經是一套「十年級」的系統。十年在機器人領域足以改朝換代，但 Cartographer 的核心設計——子地圖、分支定界、稀疏位姿調整——至今仍是 2D 光達 SLAM 的標準答案，幾乎所有後繼系統都是在這個骨架上做局部替換。本節整理近兩年的四條主要變化，讓同學知道學完本章之後，前沿在哪裡。

一、維護重心轉移：slam_toolbox 成為 ROS 2 的實務預設

最大的變化不在演算法，而在生態。隨著 ROS 2 成為主流，由 Steve Macenski 開發的 slam_toolbox 逐漸取代 Cartographer，成為多數 ROS 2 專案的預設 2D SLAM。它同樣走圖優化路線、同樣以掃描匹配建立位姿圖，但參數遠比 Cartographer 的 Lua 組態簡潔，並且內建了 Cartographer 較弱的兩項能力：地圖序列化與續建（把建好的位姿圖存檔，下次開機接著建），以及純定位模式（載入既有地圖只做定位，不再更新）。

這對教學的意涵是雙面的。一方面，若同學的目標是在 ROS 2 上把專題跑起來，slam_toolbox 通常是阻力最小的選擇；另一方面，Cartographer 仍是理解「為什麼要這樣設計」的最佳教材——它的論文把分支定界的上界推導寫得最清楚，而 slam_toolbox 的許多設計選擇，其實預設了讀者已經懂 Cartographer 在做什麼。本書的建議是：用 Cartographer 學原理，用 slam_toolbox 做專題。

二、終身建圖與多期地圖：地圖不再是一次性產物

第二條變化是問題設定本身的擴張。傳統 SLAM 假設「跑一趟、建一張圖、結束」，但真實場域會變：桌椅被移動、貨架被重排、季節改變了採光。終身建圖（lifelong mapping）處理的正是這件事——如何讓機器人在同一場域反覆行走時，把新觀測融進舊地圖，同時刪去已經不存在的障礙，而不是讓地圖愈建愈髒。slam_toolbox 的序列化與續建能力，正是朝這個方向設計的。

技術上，這把位姿圖從單一軌跡變成多期軌跡的集合，閉環不只發生在同一趟路程內，也發生在不同日期的路程之間（稱為多期 SLAM，multi-session SLAM）。這條線直接通往第六篇的多機協作：多台機器人同時建圖、再把各自的位姿圖合併，本質上就是多期 SLAM 的空間版本。

三、聯合最佳化位姿與地圖：Occupancy-SLAM 這一路

第三條是直接針對 Cartographer 架構的學理挑戰。Cartographer 的做法是「先用掃描匹配算出位姿，再把掃描插進地圖」——位姿是變數，地圖是位姿的副產品。澳洲雪梨科技大學團隊提出的 Occupancy-SLAM（發表於 Robotics: Science and Systems 2024，2025 年提出改良版）主張這個分工本身就是精度的瓶頸：既然地圖的品質取決於位姿，位姿的修正也應該回頭由地圖的一致性來驅動，那就應該把「機器人位姿」與「佔據格的每一格數值」當成同一個最佳化問題的變數，一起解。

這個想法在數學上很自然，困難在規模：一張 2D 佔據格動輒數十萬格，把每一格都當變數，問題規模比傳統位姿圖大上好幾個數量級。該團隊的貢獻正是找出這個大型問題的稀疏結構，讓它在可接受的時間內收斂。同學若讀過第 12 章的稀疏性討論（圖 12-3），會認出這是同一套思路的放大版——稀疏性永遠是大型最佳化能不能實用的分水嶺。

四、閉環偵測的學習化：從幾何比對到場景描述子

最後一條變化發生在閉環偵測。Cartographer 的分支定界是純幾何方法：它在多解析度格網上窮舉搜尋，靠上界剪枝加速。這個方法的優點是沒有訓練、沒有資料集依賴、行為完全

可預測；缺點是搜尋範圍一旦放大（例如機器人被人抱起來放到另一層樓，也就是第 11 章的綁架問題），計算量仍會爆炸。

近年的主流替代方案是「場景描述子」：把一幀掃描壓縮成一個固定長度的向量指紋，閉環偵測就變成在指紋資料庫中做最近鄰搜尋，與走了多遠無關。3D 領域的 Scan Context 系列是這條路的代表，2024 至 2025 年間的研究則延伸到以點雲密度圖做閉環、以及把地點辨識（place recognition）當成獨立問題系統性整理的綜述。2D 領域受限於單線掃描資訊量少，描述子的辨識力不如 3D，因此分支定界在 2D 仍未被取代——這也解釋了為什麼本章的技術核心到今天仍然值得細讀。

四條線索，一個共同主題

工程面（slam_toolbox）：把 Cartographer 的原理包裝得更好用，補上序列化與純定位。

問題面（終身建圖）：地圖從一次性產物變成需要長期維護的資產。

數學面（Occupancy-SLAM）：把地圖本身也拉進最佳化變數，挑戰「位姿先、地圖後」的分工。

感知面（場景描述子）：把閉環偵測從幾何窮舉換成特徵檢索，以應付大範圍與綁架問題。

共同主題：Cartographer 的骨架（子地圖 + 位姿圖 + 閉環）沒有被推翻，被替換的是骨架上的每一個零件。學會骨架，才讀得懂後面所有的零件更換。

承先啟後

本章把第 12 章的圖優化從數學式變成了一套可以在走廊上跑起來的系統。回頭看第四篇的安排：第 13 章教我們如何把掃描變成地圖，第 14 章用粒子濾波解開雞生蛋問題，第 15 章則示範了另一條路——保留歷史、容許回頭修正。圖 15-4 的數字說明了這條路在有迴路的場域中的優勢，而 15.8 節也誠實交代了它的邊界：沒有迴路，閉環就無用武之地。

從第 16 章起，鏡頭將從「演算法長什麼樣」轉向「這些演算法在真實產品裡怎麼活下來」。掃地機器人是最好的第一個案例：它便宜、量產、每天在數千萬個家庭裡運行，卻必須在極低的運算預算下完成本章談的每一件事。

公式或流程

一、局部掃描匹配的最佳化

把新掃描插入子地圖時，求一個位姿 ξ 使掃描點盡量落在子地圖的高佔據機率處：

$$\xi^* = \operatorname{argmin}_{\xi} \sum_k \left(1 - M(T_\xi \cdot h_k)\right)^2$$

其中 h_k 是掃描中的第 k 個點、T_ξ 是把它變換到子地圖座標的位姿變換（第 10 章）、$M(\cdot)$ 是該位置在機率格中的佔據機率。式子的意義是：讓所有掃描點都落在「地圖認為有障礙」的地方。這是一個非線性最小平方問題（第 12 章），以 Ceres 求解。

二、全域最佳化（稀疏位姿調整）

全域 SLAM 最佳化子地圖位姿 Ξ^m 與掃描位姿 Ξ^s，使所有約束的加權殘差最小：

$$\operatorname{argmin} \tfrac{1}{2} \sum_{ij} \rho\left(E^2(\xi_i^m, \xi_j^s; \Sigma_{ij}, \xi_{ij})\right)$$

其中 E 是殘差（第 j 筆掃描相對第 i 個子地圖的實際關係與量測關係之差）、Σ_{ij} 是該約束的共變異數（可信度，第 12.3 節）、ρ 是穩健核函數——它讓誤判的閉環約束不至於毀掉整張地圖（15.4 節）。這條式子與第 12 章的最小平方目標函數形式完全一致，只是把節點換成了子地圖與掃描。

三、分支定界搜尋

全域掃描匹配的搜尋空間定義為一個三維視窗（平移 x、平移 y、旋轉 θ）：

$$W = \{-w_x, \ldots, w_x\} \times \{-w_y, \ldots, w_y\} \times \{-w_\theta, \ldots, w_\theta\}$$

分支定界以「分支—定界—剪枝」反覆進行：

圖 15-5　分支定界閉環搜尋流程

① 預先計算多解析度格網（尺寸 1、4、16、64）

↓

② 從最低解析度開始，把搜尋空間切成數個大區塊（分支）

↓

③ 對每個區塊，在低解析度格網上計算「分數上界」（定界）

↓

④ 若某區塊的上界 < 目前最佳分數 → 整個區塊剪除，不再展開（剪枝）

↓

⑤ 否則把該區塊再切細，到更高解析度重複 ②–④

↓

⑥ 抵達完整解析度且分數超過門檻 → 接受為閉環約束

四、Cartographer 完整資料流

圖 15-6　Cartographer 端到端流程

感測器：雷射掃描 + IMU +（可選）里程計

↓

【局部 SLAM】掃描匹配 → 插入當前子地圖 → 子地圖滿了則封存，開新的

↓　（產生局部軌跡，會漂移）

【全域 SLAM，背景執行緒】

取樣節點 → 對既有子地圖做分支定界匹配 → 找到閉環約束

↓

組成位姿圖（子地圖位姿 + 掃描位姿）

↓

每隔數秒以 Ceres 執行稀疏位姿調整（含穩健核函數）

↓

輸出：全域一致的地圖與軌跡（閉環時整張地圖被拉正）

案例

案例 15-1　閉環的那一刻：課堂上最震撼的三秒鐘

第 12 章案例 12-1 描述過閉環拉正地圖的畫面，而 Cartographer 讓這個畫面成為課堂上可以重現的實驗。

典型的教學場景是這樣的：同學遙控機器人繞行系館一圈。前半程一切正常，地圖工整；繞到後半段時，牆線開始與最初建的部分錯開，走廊出現明顯的重影，同學臉上開始出現疑惑。就在機器人回到起點附近的瞬間——背景執行緒找到了閉環約束，全域最佳化執行——整張地圖在螢幕上「唰」地一下重組，重影合而為一，錯開的走廊接合，軌跡形成一個漂

亮的封閉迴路。

這三秒鐘，把第 12 章的圖優化、第 15.4 節的分支定界與稀疏位姿調整，全部濃縮成一個看得見的瞬間。教學經驗顯示，親眼看過這一幕的同學，對「為什麼要用圖優化」的理解遠勝於任何講述。建議教師務必安排足夠的空間讓機器人繞完一個完整迴路——這是本章最值得投資的課堂時間。

案例 15-2　Google 為何要做 Cartographer：室內地圖的野心

Google 開發 Cartographer 並非純學術興趣。當時 Google 已經用街景車繪製了全世界的道路，但室內空間——商場、機場、車站、博物館——仍是地圖的空白區。要繪製室內地圖，傳統測繪太慢太貴，必須有能「邊走邊建圖」的技術。

這解釋了 Cartographer 的幾個設計取向：為什麼要能即時（操作員需要立即知道哪裡還沒掃到）、為什麼要能處理大場景（整座商場）、為什麼要做背包式系統（人能走的地方都要能掃）、以及為什麼閉環如此重要（大範圍行走必然累積漂移）。

這個案例給同學一個重要啟示：技術的設計取向，往往由應用需求決定。Gmapping 出自學術實驗室，追求演算法的優雅；Cartographer 出自產業需求，追求大規模場域的實用性。兩者沒有優劣，只有適不適合——這正是第 8 章「感測器選型是綜合判斷」觀念的再次印證。

案例 15-3　感測器組態的實驗：什麼配置最適合什麼場域

有研究團隊系統性地測試了不同感測器組合對 Cartographer 效能的影響，結果頗具啟發性：在雜亂的房間中，三具 2D 光達加上 IMU 與里程計的組合誤差最低；而在長走廊場景中，單具 3D Velodyne 光達加 IMU 則表現最佳。

這個結果與本書前面的討論完全呼應。雜亂房間中，多具 2D 光達從不同角度掃描，能涵蓋更多幾何特徵、減少遮蔽（第 8 章的高度盲區）；而長走廊的困境在於縱向缺乏約束

（第 13.6 節），3D 光達能看到天花板與地面的結構變化，正好補上這個缺口。

對大學專題與大學部實作的啟示是：不要盲目追求「最貴的感測器」，而要依場域特性選擇組態。這也是一個絕佳的專題題目——在校園不同場域（教室、走廊、中庭）實測不同感測器組合的建圖品質，用第 13.7 節的方法量化比較。這樣的研究既有教學價值，也有實用意義。

實作活動

實作 15-1　Cartographer 建圖與閉環觀察

目的：執行 2D Cartographer 建圖，親眼觀察子地圖的形成與閉環拉正的瞬間，並與第 14 章的 Gmapping 做對照實驗。這是第四篇最重要的一個實作。

第一部分：模擬建圖與閉環觀察（全班可做）

27. 在 TurtleBot3 Gazebo 模擬環境（第 28 章）中安裝並啟動 Cartographer 的 2D 設定。
28. 在 RViz 中顯示子地圖與軌跡，先在小範圍移動，觀察子地圖如何一塊塊形成、如何部分重疊。
29. 遙控機器人繞行模擬世界一個完整迴路，全程注意觀察：前半段地圖工整、後半段開始出現錯位。
30. 關鍵時刻：回到起點附近時，留意閉環是否觸發、地圖是否被拉正。錄下這個瞬間並在課堂上重播討論。
31. 若閉環未觸發，嘗試調整閉環分數門檻與取樣比例（表 15-1），再試一次。

第二部分：對照實驗——Cartographer vs Gmapping（核心）

32. 在同一個模擬世界、同一條行走路線下，分別用 Gmapping（第 14 章）與 Cartographer 各建一次圖。

33. 用第 13.7 節的方法評估兩張地圖：牆是否筆直？迴路是否閉合？有無重影？量測地圖上的關鍵尺寸與模擬設定值比較。

34. 把兩張地圖並排，標示出差異最明顯的區域，並解釋原因（提示：表 15-2）。

35. 觀察資源用量：比較兩者的 CPU 與記憶體使用情形，對照第 14 章公式五與 15.2 節的討論。

第三部分：實體平台建圖（有硬體的班級）

36. 準備 Lua 設定檔，依你的平台調整（有無 IMU、有無里程計、光達型號與安裝位置）。

37. 先在教室建圖驗證設定正確，再挑戰較大場域——理想的是能繞一個完整迴路的走廊或系館一層樓。

38. 緩慢行走，務必走完整個迴路回到起點，觀察閉環。

39. 匯出地圖並評估品質，與同一場域的 Gmapping 結果比較。

討論問題

9. 你觀察到閉環拉正的瞬間了嗎？拉正前後，地圖上哪些區域變化最大？為什麼是那些區域？

10. 對照實驗中，Cartographer 與 Gmapping 的地圖在哪些方面差異最明顯？這印證了表 15-2 的哪幾項？

11. 如果場域沒有迴路（例如只是一條走到底的走廊），Cartographer 的閉環機制還能發揮作用嗎？此時它相對 Gmapping 還有優勢嗎？

12. （大學部）15.3 節提到閉環取樣比例「太低錯過閉環、太高拖慢即時性」。請實際調整這個參數，記錄閉環偵測成功率與系統延遲的變化，找出你場域的合適值。

延伸挑戰（大學部）

其一：設計一個「錯誤閉環」實驗——在模擬世界中建造兩個外觀幾乎相同的房間，觀察

Cartographer 是否會誤判閉環；若誤判，觀察穩健核函數是否減輕了災難程度。這將深化你對 15.4 節穩健化的理解。其二：若學校有 3D 光達，嘗試 Cartographer 3D 模式，掃描一間實驗室或一段樓梯，產生三維點雲模型，體驗背包式建模的概念（15.6 節），並為第五篇（第 17、18 章）的 3D SLAM 預作準備。

延伸閱讀

15. W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM," ICRA, 2016.（Cartographer 的原始論文，本章主要依據）
16. Cartographer 官方文件（google-cartographer.readthedocs.io）與 Cartographer ROS 文件.（含 Algorithm walkthrough for tuning 調參指南，實作必讀）
17. Google Open Source Blog, "Cartographer" 發布公告.（說明開發動機與室內測繪的應用背景）
18. J. Elseberg and A. Nüchter, "Improving Google's Cartographer 3D Mapping by Continuous-Time SLAM," ISPRS Archives, 2017.（3D 模式的改良研究，含背包式系統的實測）
19. slam_toolbox 開源專案（GitHub）.（ROS 2 環境中另一套基於圖優化的 2D SLAM，可與 Cartographer 對照）
20. Ceres Solver 官方文件（ceres-solver.org）.（Cartographer 使用的最佳化函式庫，呼應第 12 章）
21. Cyrill Stachniss, Graph-Based SLAM 系列課程影片（YouTube）.（圖優化 SLAM 的理論基礎，搭配本章閱讀效果佳）

小結

15. Cartographer 由 Google 開發、2016 年開源，核心特色是「即時閉環」；它分為兩個同時運行的子系統：局部 SLAM（前端，建立子地圖，局部一致但會漂移）與全域 SLAM（後端，在背景尋找閉環約束並最佳化位姿圖）。
16. 子地圖是由數十筆掃描組成的機率格小地圖；分子地圖的三個理由是限制誤差累積、提供最佳化的單元（節點數大減故能即時）、以及成為穩定的閉環比對對象。記憶體與場景大小成正比，故能處理整棟建築。
17. 系統中有兩種掃描匹配：局部匹配把新掃描插入當前子地圖（以 Ceres 求解非線性最小平方）；全域匹配則搜尋閉環，難度高得多，需限制取樣節點數以兼顧閉環成功率與即時性。
18. 分支定界法是即時閉環的關鍵：在多解析度預算格上，由低解析度算出分數上界，將不可能勝出的區塊整片剪除，使搜尋量大幅下降；找到約束後以稀疏位姿調整拉正全圖，並用穩健核函數防止錯誤閉環造成災難。
19. 2D 實作需備妥雷射、TF 與（建議）IMU;Cartographer 以 Lua 設定檔組態，參數較多，關鍵在子地圖掃描數、解析度、閉環取樣比例與分數門檻；務必走完整迴路以觀察閉環。
20. 3D 模式需 3D 光達與 IMU，子地圖為體素格，運算量大；背包式空間建模讓操作員邊走邊建立整棟建築的三維模型，應用於 BIM、古蹟保存、設施管理，是大學場域極具潛力的專題方向。
21. 與 Gmapping 相比：Cartographer 基於圖優化而非粒子濾波、以子地圖而非每粒子一張地圖、閉環為核心功能、記憶體與場景成正比、支援 3D 與 IMU，但參數較複雜。兩者正是第 11、12 章兩條理論路線的實體化身。

第四篇的兩大技術主角至此完備。下一章，我們把這些技術帶回最貼近生活的場景——掃地機器人。從隨機碰撞到 SLAM 規劃式清掃、從房間分割與禁區設定到覆蓋路徑規劃，並誠實面對家庭場景的真實挑戰：桌椅腳、鏡面、低矮障礙物與寵物。你會發現，本篇所有

理論都在你家客廳裡默默運作著。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 16 章

掃地機器人 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

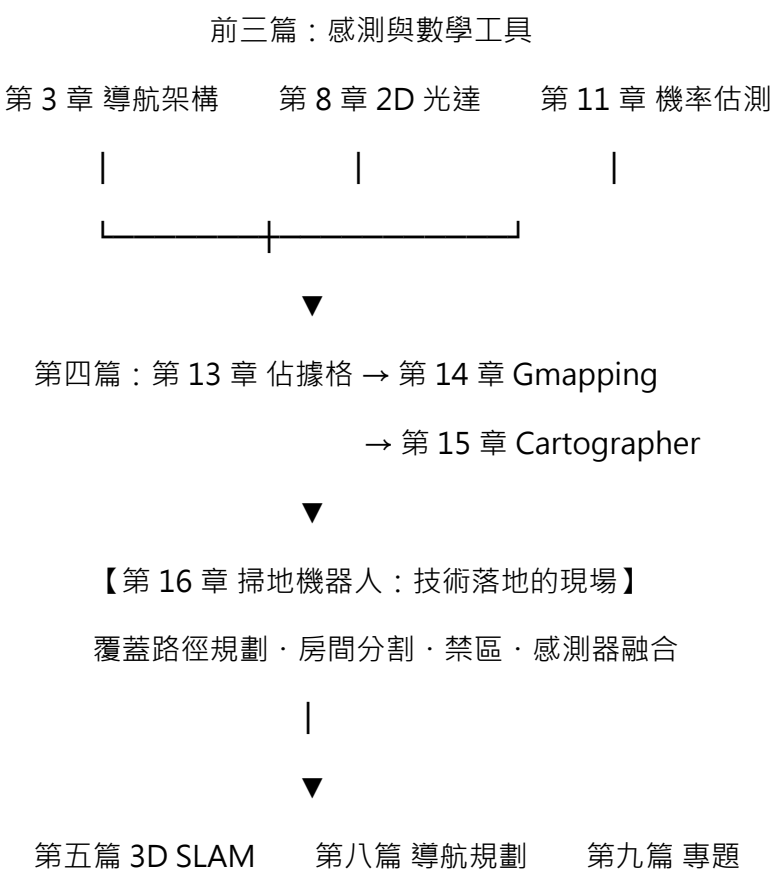
大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

本章是第四篇的收尾，也是全書第一次把累積了十五章的技術，整組放進一台你買得到、摸得到的產品裡。前三章談的是演算法本身——佔據格怎麼建、粒子怎麼濾、位姿圖怎麼解；本章問的是另一個問題：當這些演算法必須在一台兩萬元、運算晶片只有手機等級、每天在陌生家庭裡獨自工作八小時的機器上運行時，會發生什麼事。答案往往不是「演算法不夠好」，而是「真實世界比論文裡的走廊複雜得多」。

所以本章的閱讀方式與前三章不同。前三章是由下而上建立工具，本章是由上而下拆解一個產品：看它用了哪些工具、在哪裡妥協、又用什麼方法補救。讀完之後回頭看第 8 章的角解析度、第 13 章的地圖病徵、第 14 章的打滑問題，會發現那些當時看起來抽象的限制，每一條都對應到你家掃地機器人的某個具體行為。

圖 16-0　第 16 章的連結地圖



學習目標

第四篇的理論走到這裡，終於要回到最貼近生活的場景——你家客廳。掃地機器人是 SLAM 技術最成功的大眾化案例，前面十五章的所有觀念，幾乎都能在這台小小的圓盤機器上找到對應。本章是第四篇的收尾，也是一堂「把理論帶回家」的課。學完本章之後，同學應能夠達成下列目標：

21. 說明掃地機器人為什麼需要 SLAM，以及沒有 SLAM 會有什麼後果。
22. 比較隨機碰撞式與 SLAM 規劃式清掃的差異，並理解覆蓋率與效率的量化意義。
23. 說明光達掃地機器人如何建立房間地圖，並連結第 13 至 15 章的技術。
24. 說明房間分割、禁區設定與覆蓋路徑規劃的原理。
25. 列舉掃地機器人的感測器組合，說明各自的角色與互補關係。
26. 分析家庭場景的真實挑戰：桌椅腳、鏡面、低矮障礙物與動態環境。
27. 從掃地路線理解覆蓋路徑規劃的基本策略與評估指標。

核心概念

中文名詞	英文名詞	概念說明
覆蓋路徑規劃	Coverage Path Planning	規劃路徑走遍區域中每一個可通行位置。
弓字形路徑	Boustrophedon Path	像犁田般來回折返的清掃路線，是覆蓋規劃的基本形式。
隨機碰撞式	Random Bounce	早期掃地機的策略：撞到障礙就轉向，無地圖無規劃。
覆蓋率	Coverage Rate	實際清掃到的面積佔可清掃面積的比例。
重複率	Redundancy Rate	重複清掃同一位置的比例，越低效率越高。
房間分割	Room Segmentation	把整張地圖自動切分成有意義的房間區塊。
禁區	No-go Zone	使用者在地圖上劃定、機器人不得進入的區域。

中文名詞	英文名詞	概念說明
虛擬牆	Virtual Wall	地圖上的虛擬界線，功能與禁區類似。
懸崖感測器	Cliff Sensor	偵測前方地面突然消失（樓梯）的紅外線感測器。
3D 飛時測距	3D ToF	以立體方式量測前方障礙形狀的感測器。
結構光	Structured Light	投射光線圖案以判讀障礙物形狀與距離。
多樓層地圖	Multi-floor Map	記憶多個樓層地圖並自動辨識當前所在樓層。

16.1 掃地機器人為什麼需要 SLAM

這個問題乍看簡單：不就是把地板掃乾淨嗎？但仔細想想，「把整個家的地板都掃過一遍、而且不重複太多次」這件事，其實需要回答第 3 章的導航三問：我在哪裡？我要去哪裡？我該怎麼去？

沒有 SLAM 的三個困境

- 不知道哪裡掃過了：沒有地圖與定位，機器人無法記錄「我已經走過這一塊」。結果是有些地方掃了五次、有些地方一次都沒到。
- 不知道何時算結束：沒有地圖就不知道房子有多大、還剩多少沒掃。早期機型只能靠計時器——掃滿三十分鐘就收工，完全不管掃完了沒。
- 不知道怎麼回家：電量將盡時要回充電座，沒有地圖就只能亂走碰運氣，常常卡在半路沒電。

SLAM 帶來的四項能力

26. 系統性清掃：有了地圖，才能規劃弓字形路線逐區覆蓋，大幅提升覆蓋率、降低重複率（16.2 節）。
27. 斷點續掃：電量不足時回充，充飽後能回到「上次掃到的地方」繼續——這需要準確的定位與記憶的地圖。
28. 使用者互動：App 上顯示房間地圖，讓使用者能指定「只掃廚房」、設定禁區、指哪掃哪。這是地圖帶來的產品價值。
29. 多次清掃的一致性：第二次清掃不必重新探索，直接載入既有地圖定位即可（第 14 章案例 14-3 的 AMCL）。

把前面的章節串起來

掃地機器人幾乎是本書前十五章的實體總複習：它用第 8 章的 2D 光達感知環境、第 13 章的佔據格地圖記錄空間、第 14 或 15 章的 SLAM 演算法同時定位與建圖、第 10 章的座標

轉換整合感測器、第 11 章的粒子濾波做重定位（綁架問題，第 3 章）。理解掃地機器人，就是理解整個第四篇。

16.2 隨機碰撞式掃地 vs SLAM 規劃式掃地

掃地機器人的演進史，是一部從「反應式」走向「審議式」的縮影（第 3 章的架構討論）。

隨機碰撞式：反應式架構的極致

2002 年 iRobot 推出第一代 Roomba 時，採用的是隨機碰撞策略：直線前進、撞到東西就轉一個隨機角度、繼續前進。搭配螺旋、沿牆等幾種簡單行為模式交替使用。

這個設計的靈感其實來自第 3 章提過的行為式架構——不建模、不規劃，讓感知直接驅動行為。它的優點是極其便宜、程式簡單、不會因為地圖錯誤而失效；缺點也很明顯：路線雜亂、覆蓋不均、耗時極長。

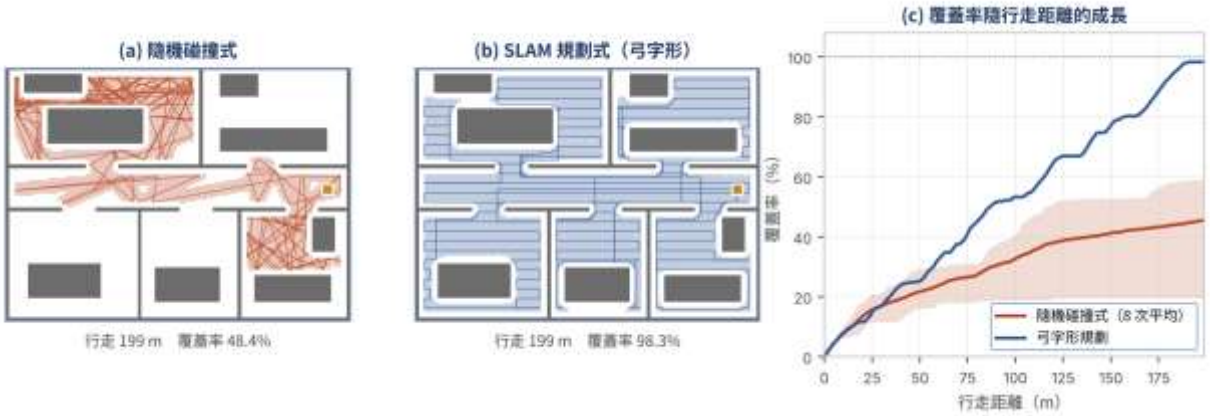
有趣的是，這個策略在數學上並非全無道理：只要時間夠久，隨機行走理論上能覆蓋整個連通區域。問題是「夠久」可能意味著一小時——而且永遠無法保證真的掃完了。

SLAM 規劃式：審議式架構的勝利

2010 年 Neato 首度把 2D 光達裝上掃地機，讓機器人能建立佔據格地圖並規劃系統性路線。現代機型不再只是靠隨機碰撞轉向的旋轉刷具，而是執行裝置端 AI 來建立地圖、避開障礙、並適應真實的居家生活方式。

規劃式清掃的核心是弓字形路徑（Boustrophedon，原意為「像牛犁田般來回」）：機器人沿著一個方向直線清掃到底，轉 180 度移動一個刷寬，再反向掃回來，如此往復覆蓋整個區域。

圖 16-1　同一間公寓、同樣走 199 公尺：兩種清掃策略的結果



這張圖用一次對照實驗，把表 16-1 的文字差異變成可以量的數字。場景是一間 9.0×6.6 公尺的三房公寓，含主臥、次臥、書房、客廳、廚房與走道，並擺了床、沙發、書桌、流理台等家具；扣掉機器人半徑無法進入的牆邊與家具邊，可清掃面積為 27.5 平方公尺。兩種策略都從右側的充電座（金色方塊）出發，都走 199 公尺——也就是說，電池消耗是一樣的。

圖(a)是隨機碰撞式：直線前進、撞到就轉一個隨機角度。路徑呈現典型的亂麻狀，而結果是只掃到 48.4%。更值得注意的是它掃到了哪裡——主臥、走道與廚房有明顯覆蓋，但次臥、書房與客廳幾乎是空白。原因很具體：門口只有 85 公分寬，扣掉機器人半徑後可通行的縫隙不到 60 公分，一個隨機方向要恰好穿過這道縫，機率很低。隨機策略不是「慢慢來總會掃完」，而是會被困在它最先進入的那幾個房間裡。

圖(b)是 SLAM 規劃式：先用地圖把空間切成房間（下一節的圖 16-2），再在每個房間裡走弓字形，房間之間以最短路徑連接。同樣 199 公尺，覆蓋率 98.3%。路徑圖上一條條整齊的來回線，就是「弓字形」這個名字的由來；家具周圍那一圈沒掃到的白邊，則是機器人半徑造成的物理極限——這也解釋了為什麼實務上覆蓋率很難真正到 100%。

圖(c)把覆蓋率隨行走距離的成長畫出來，是本圖最有訊息量的一格。紅色實線是八次隨機實驗的平均，淡紅色帶是八次的最好與最壞範圍。兩條線在前 25 公尺幾乎重疊——這很合理，剛開始時到處都是沒掃過的地方，亂走也會有收穫。但從約 40 公尺之後開始分岔：弓字形持續線性爬升直到接近 100%，隨機式則快速趨於平緩，八次實驗的最終覆蓋率落在 19.5%

到 59.0% 之間。

那條淡紅色帶的寬度，比平均值本身更值得討論。它代表的是「同樣一台機器、同樣的程式、同樣的時間，今天掃完的結果和明天可能差三倍」。對一個每天都要用的家電來說，不可預測比效率低更致命——使用者無法判斷它到底掃完了沒。這正是 16.1 節「不知道何時算結束」那個困境的量化版本，也是 SLAM 為掃地機器人帶來的最大價值：不只是掃得更乾淨，而是掃得可以預期。

關於這組模擬的說明

這是教學用的簡化模擬：機器人視為一個直徑 26 公分的圓，有效刷寬 26 公分，不含邊刷的額外涵蓋。

隨機式的實作為「直線前進、碰撞後轉隨機角度」。真實的 Roomba 還會穿插螺旋與沿牆模式，覆蓋率會略高於本圖，但多房間住宅中被困住的結構性問題一樣存在。

弓字形規劃走了 199 公尺，理論最短覆蓋距離為 27.5 ÷ 0.26 ≈ 105.6 公尺，因此重複率約 1.88。這個數字在本章的評估指標裡屬於中等——轉行、繞家具與房間之間的移動都算在內。

同學可在實作 16-1 第三部分自行重現這張圖，並試著把重複率壓到 1.5 以下。

比較項目	隨機碰撞式	SLAM 規劃式
是否建圖	否	是（佔據格地圖）
清掃路線	隨機、雜亂	弓字形、系統性
覆蓋率	不保證，依賴時間	高，可接近完全覆蓋
重複率	高	低
所需時間	長	短
能否斷點續掃	否	是

比較項目	隨機碰撞式	SLAM 規劃式
成本	極低	較高（需光達或相機）
對地圖錯誤的容忍	不受影響	地圖錯則清掃錯

表 16-1　兩種清掃策略的比較

注意最後一列：規劃式並非全無缺點。若 SLAM 建圖失敗（第 13.7 節的地圖病徵），機器人會照著一張錯誤的地圖清掃，反而可能漏掉整個房間。這呼應第 13 章的結論：地圖品質直接決定系統成敗。

16.3　光達掃地機器人與 2D 光達建圖

第 1 章案例 1-1 提過，2017 年小米以親民價格推出光達掃地機器人，讓 SLAM 技術首次大量進入一般家庭。本節深入看它如何運作。

硬體配置

典型的光達掃地機器人在機頂有一個凸起的圓塔，裡面是一顆旋轉的 2D 光達（第 8 章），掃描高度約在機身頂部（離地 8 至 10 公分）。它每秒旋轉數圈，量測周圍數百個方向的距離。

值得注意的是成本控制：家用掃地機的光達多採低成本的三角測距原理，量測範圍約 6 至 8 公尺、精度數公分——遠不如工業型號，但對室內建圖已經足夠。這正是第 8 章「解析度要與應用需求匹配」的實例。

建圖流程：第 13 至 15 章的實踐

30. 首次進入陌生環境：機器人執行 SLAM，一邊移動一邊建立佔據格地圖並估計自己的位姿（第 13 章）。多數機型會先沿牆走一圈建立房屋輪廓，再處理內部。

31. 演算法選擇：早期機型多用粒子濾波類方法（第 14 章），近年因運算晶片進步，越來越多採用圖優化與閉環偵測（第 15 章），使大坪數住宅的地圖更一致。

32. 地圖儲存：建好的地圖存在機器內部與雲端，日後清掃直接載入。

33. 後續清掃：載入地圖後執行定位（AMCL 類方法，第 11 章），不需重新建圖，可直接規劃路線。

近年趨勢：光達與視覺並用

現代高階機型已不只倚賴光達。業界的做法是雷射光達負責建立地圖與追蹤機器人位置，AI 相機則負責視覺 SLAM 的部分——辨識地板上的物體並決定如何處理。這正是第 9 章「光達給幾何、相機給語意」的商業實踐。

這個組合直接呼應第 8 章表 8-2 的互補分析：光達不怕黑暗、幾何精確，但不知道「那是什麼」；相機能辨識物體，卻在昏暗與無紋理處失效。因此有廠商在相機旁加裝 LED 燈以維持在黑暗或低矮空間的視覺能力。

16.4 房間分割、禁區設定、路徑覆蓋

有了地圖只是第一步。要讓地圖真正好用，還需要三項功能，它們共同構成了使用者眼中的「智慧」。

一、房間分割

房間分割是把一整張佔據格地圖，自動切分成「客廳、臥室、廚房」等有意義的區塊。這對使用者體驗至關重要——沒有分割，使用者只能說「掃全部」；有了分割，才能說「只掃廚房」。

分割的基本思路是找出空間的「瓶頸」：門口與狹窄通道處，自由空間的寬度會突然變窄，這些位置就是天然的分界。常用方法包括依據距離變換找出房間的「核心區」再向外擴張、或是偵測門框的幾何特徵。

現代機型建好地圖後，能為房間貼標籤、排除禁區，甚至記住哪些區域需要更頻繁的清掃。這已經觸及第 3 章提過的語意地圖概念——機器人不只知道「這裡可通行」，還知道「這裡是廚房」。第 32 章會完整討論如何用 AI 做語意標註。

圖 16-2 房間分割：用距離變換找出門口這個「瓶頸」



房間分割聽起來像是一件需要人工智慧的事——畢竟「哪裡是臥室」是個語意問題。但這張圖顯示，只用兩步純幾何運算就能把它做得相當好。圖(a)是 SLAM 建好的佔據格地圖，此時它只是一堆黑白格子，沒有任何「房間」的概念。

圖(b)是關鍵的一步：距離變換。它替每一個自由格算出「這一格離最近的牆有多遠」，顏色越亮代表越空曠。看圖就能發現一個規律——房間中央是亮的（離牆遠），而門口是暗的一道窄縫（兩側都是牆，離牆都近）。紅色三角標出的五個位置就是這間公寓的門口，它們在距離變換圖上全部表現為「瓶頸」。白色等高線畫的是 0.45 公尺這個門檻：門口的距離值約 0.42 公尺，恰好被切斷，於是各房間的核心自然分離成獨立的區塊。

圖(c)是第二步：以這些核心為種子做分水嶺（watershed）擴張，讓每個核心沿著距離變換的梯度長回去，直到在門口處彼此相遇為止。結果是六個區塊，恰好對應主臥、次臥、書房、走道、客廳、廚房，連面積都一併算出來了。整個過程沒有用到任何機器學習，只用了距離變換與分水嶺——兩個在 OpenCV 裡各只需一行的運算。

這個結果對產品的意義很直接：有了分割，App 上才能顯示「客廳 3.8 平方公尺」並讓使用者點選「只掃廚房」；有了分割，覆蓋規劃才能逐房間進行（圖 16-1(b) 正是這樣做的），

而不是在整個平面上硬切長條。不過也要誠實說，純幾何方法有它的極限：開放式廚房與客廳之間沒有門，就沒有瓶頸可找，這類格局往往被切成一塊或切錯位置——這正是現代機型開始引入語意資訊的理由。

距離變換門檻怎麼挑

門檻的物理意義是「房間核心至少要離牆這麼遠」。它必須大於門口半寬、小於房間半寬。

本圖門寬 0.85 公尺（半寬約 0.42），房間最窄處半寬約 0.6 公尺以上，因此 0.45 公尺是安全的中間值。

門檻太低：門口沒被切斷，相鄰房間連成一塊（本章模擬在 0.40 以下就會發生）。

門檻太高：房間核心被家具切碎，一個房間被分成好幾塊，必須再靠最小面積過濾合併。

實務上門寬相當標準（台灣住宅多為 80～90 公分），所以這個門檻可以寫死；真正難處理的是開放式格局與無門的穿堂。

二、禁區與虛擬牆

禁區（No-go Zone）讓使用者在 App 地圖上劃出機器人不得進入的區域——寵物碗、小孩的積木區、易被纏繞的電線區。虛擬牆則是劃一條線禁止跨越。

技術上，這是在代價地圖（第 3 章）上人為加入「障礙」：規劃路徑時把禁區視為不可通行。這是一個聰明的設計——不需要改動 SLAM 或感測器，只需在規劃層加一層使用者定義的約束。同學可以想想：這其實就是把「使用者的知識」注入機器人系統的一種方式。

三、覆蓋路徑規劃

覆蓋路徑規劃（Coverage Path Planning,CPP）與一般路徑規劃有本質差異：

- 一般路徑規劃（第 3、29 章）：求「從 A 到 B 的最短路徑」，目標是抵達。

- 覆蓋路徑規劃：求「走遍每一個可通行位置的路徑」，目標是不遺漏。

掃地機器人是後者的典型應用。基本策略是把地圖分割成無障礙的長條區域，在每個區域內走弓字形，再依序連接各區域。機器人利用機載感測器與（若有的話）預先載入的樓層平面圖輪廓，規劃一條能抵達每個可進入房間、走廊與空間的覆蓋路徑。

評估指標

- 覆蓋率：實際掃到的面積 ÷ 可清掃面積。理想值接近 100%，但家具下方、角落往往難以完全覆蓋。
- 重複率：總行走距離 ÷ 理論最短覆蓋距離。越接近 1 越有效率。
- 清掃時間：完成全屋所需時間，是使用者最有感的指標。

這三個指標互相拉扯：提高覆蓋率往往增加重複率與時間。工程上的取捨，正是產品設計的核心。這也是第 31 章專題評分 Rubric 可以參考的量化思路。

16.5 掃地機器人的感測器：光達、IMU、碰撞、懸崖感測器

一台掃地機器人身上的感測器，幾乎是本書第二篇的濃縮版。本節逐一說明它們的角色與互補關係。

感測器	角色	對應章節
2D 光達	建圖與定位的主力，量測周圍距離	第 8 章
IMU	偵測轉向與傾斜，輔助位姿估計	第 7 章
輪速編碼器	提供里程計，SLAM 的運動預測來源	第 3、13 章
碰撞感測器	光達沒看到時的最後防線	本章
懸崖感測器	向下紅外線，防止從樓梯跌落	本章
紅外 / 超音波	近距離障礙偵測，補足光達盲區	第 8 章
AI 相機	辨識物體與語意，避開電線與排泄物	第 9、32 章

感測器	角色	對應章節
3D ToF / 結構光	量測前方障礙的立體形狀	第 8、9 章

表 16-2 掃地機器人的感測器組合

為什麼需要這麼多感測器

答案是第 8 章案例 8-1 的「高度盲區」：2D 光達只掃描單一水平高度切面，看不到低於或高於它的東西。因此：

- 低矮障礙（電線、拖鞋、寵物玩具）：在光達掃描平面之下，完全看不到，必須靠前方的紅外線、3D ToF 或相機。
- 懸空障礙（懸掛的窗簾、低矮家具邊緣）：可能剛好在掃描平面上下，偵測不穩定。
- 樓梯與落差：光達水平掃描完全無法偵測「地面突然消失」，必須靠向下的懸崖感測器——這是掃地機器人不可或缺的安全裝置。
- 玻璃與鏡面：第 8 章的老問題，雷射穿透或反射，需靠碰撞感測器兜底。

圖 16-3 高度剖面：2D 光達看不到的那些東西

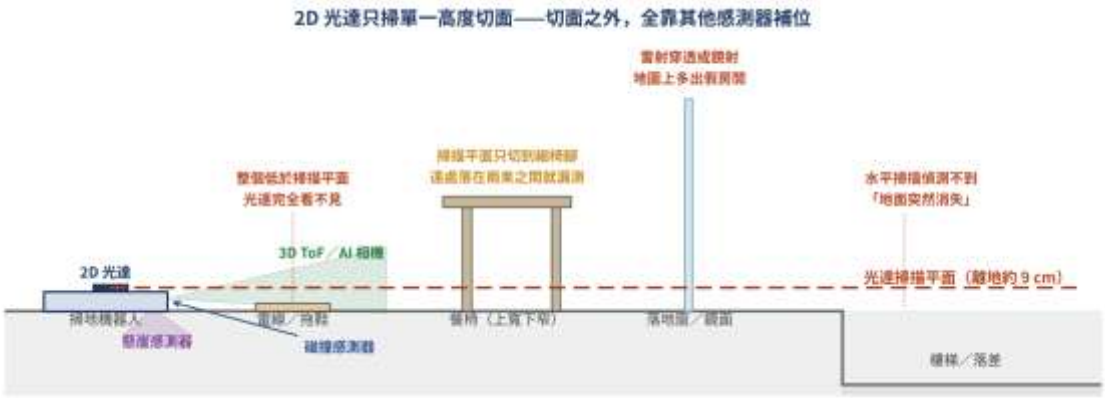


表 16-2 列了八種感測器，初學者的第一個疑問通常是：裝這麼多做什麼？這張側視剖面圖就是答案。紅色虛線是 2D 光達的掃描平面，離地約 9 公分——注意它是一個「平面」，不是一個視野。光達對這個平面上的東西瞭若指掌，對平面之外的一切完全無知。本書第 8 章稱之為高度盲區，而家庭正是高度盲區最致命的場所。

從左往右看四種典型的失效。電線與拖鞋高度只有三、四公分，整個落在掃描平面之下，光達不只是看不清楚，而是根本不知道它們存在——這就是掃地機器人纏住電線、拖著襪子跑遍全屋的物理原因。餐椅則是另一種失效：它上寬下窄，掃描平面只切到細細的椅腳，而第 8 章的角解析度公式告訴我們，距離夠遠時相鄰兩束雷射的間隔會超過椅腳寬度，整根椅腳就落在兩束之間被漏掉。案例 16-2「為什麼你家的掃地機器人總是撞到那張椅子」，講的就是這個幾何事實。

落地窗與鏡面是第 8 章反覆強調的光學難題：雷射要嘛直接穿透、要嘛被鏡面反射到別的方向，結果是地圖上出現一個根本不存在的房間，或是一道看不見的牆。樓梯與落差則更危險——水平掃描完全無法偵測「地面突然消失」，若沒有別的感測器，機器人會直接走下樓梯。

圖中三個彩色區塊就是補位的答案，而它們的擺放位置本身就在說明設計邏輯：3D ToF 與 AI 相機朝前上方張開一個錐狀視野，補的是掃描平面「以下、以上」的立體形狀；懸崖感測器朝正下方，補的是地面本身；碰撞感測器貼在機身前緣，是最後一道防線——當前面所有感測器都失效時，至少還有物理接觸。這三層由遠而近、由精確到粗略的配置，正是第二篇感測器融合思想最生活化的實例：沒有全能的感測器，只有恰當的分工。

一個可以自己做的觀察

把一本書平放在地上，看你家掃地機器人是否避開——書厚約 3 公分，低於掃描平面。

換成一個紙箱（高度超過 10 公分），通常會被乾淨地繞開——因為它進入了掃描平面。

這個對照實驗只需要兩樣家裡現成的東西，卻能讓同學親眼確認「掃描平面」是真的存在的一條線。

延伸觀察：把書換成細長的椅腳或立燈底座，從不同距離放置，測試第 8 章角解析

度的影響。

現代高階配置

近年高階機型的感測器組合越來越豐富。有機型在單一 AI 相機之外搭配 3D 結構光，構成能辨識大量常見障礙物類型的進階障礙偵測系統。不同感測器各司其職：LED 燈維持相機在黑暗或低矮空間的視力、3D 結構光讀取正前方障礙的形狀、3D ToF 則同時量測高度與寬度並能在強光下維持效能。

這個演進趨勢再次印證本書的核心信念：沒有全能的感測器，只有恰當的組合。掃地機器人正是感測器融合（第二篇）最貼近生活的教材。

16.6 家庭場景中的 SLAM 挑戰：桌椅、鏡面、低矮障礙物

實驗室的教室是 SLAM 的友善場景（第 13.6 節），但家庭是另一回事。本節誠實面對真實家庭的挑戰——這也是同學觀察自家掃地機器人時最能印證的部分。

一、細瘦的桌椅腳

第 8 章案例 8-1 詳細討論過這個問題：餐椅腳細瘦，在遠處可能整個落在兩束雷射之間而被漏測（第 8 章的角解析度公式）。結果是地圖上沒有這根椅腳，機器人一頭撞上去。

- 對策：近距離時角解析度足夠，加上碰撞感測器兜底；現代機型用 3D ToF 或相機在近距離補足偵測。

二、鏡面與落地窗

鏡子會讓光達「看到」鏡中的虛像，在地圖上畫出一個根本不存在的房間；落地窗則讓雷射直接穿透，機器人以為那裡可以通行。這是第 8 章反覆強調的光達弱點，在家庭中極為常見。

- 對策：使用者手動在 App 中設定禁區（16.4 節）；部分機型用超音波偵測玻璃（聲波會被玻璃反射）；建圖時暫時遮蔽鏡面。

三、低矮與動態障礙

地上的電線、襪子、寵物玩具、寵物排泄物，是掃地機器人最著名的災難來源。它們低於光達掃描平面，且位置每天都不同——地圖上不會有它們。

- 對策：這正是 AI 相機的價值所在。感測系統融合光達、相機、有時還有結構光感測器的資料，以偵測牆壁、家具腳、玩具甚至電線，而且這個過程是持續進行的。

四、動態環境：家會變

椅子被拉開、行李箱放在客廳、門開了又關——家庭環境每天都在變。這對 SLAM 是嚴峻考驗，因為地圖需要既「記得」永久結構（牆），又能「忘記」臨時物件（第 13.5 節的地圖可塑性）。

- 對策：對佔據機率設上下限讓地圖保持可塑性（第 13.5 節）；區分「永久地圖」（牆與家具）與「臨時障礙層」（當次清掃偵測到的物體）；部分機型會在多次清掃後統計哪些障礙是常駐的。

五、地面材質變化

磁磚換地毯時，輪子的摩擦力與滑動特性改變，里程計品質下降（第 14.6 節的打滑問題）；地毯還會讓機器人「陷入」而改變機身高度，影響光達的掃描平面。

- 對策：調高 SLAM 對里程計的雜訊假設、多倚賴掃描匹配；部分機型會偵測地毯並調整行為（如提高吸力、抬高拖布）。

家庭挑戰	根本原因	主要對策
細瘦桌椅腳	角解析度不足，遠處漏測	近距離偵測、碰撞感測器、3D ToF

家庭挑戰	根本原因	主要對策
鏡面與落地窗	雷射穿透或反射，產生虛像	使用者設禁區、超音波、暫時遮蔽
低矮動態障礙	在光達掃描平面之下	AI 相機辨識、3D 結構光
環境每天在變	地圖需兼顧記憶與遺忘	機率設限、分層地圖、多次統計
地毯與地面變化	里程計品質下降、機身高度改變	調高里程計雜訊、倚賴掃描匹配

表 16-3　家庭場景的 SLAM 挑戰與對策

16.7　教學活動：從掃地路線理解 Coverage Path Planning

本節說明如何把覆蓋路徑規劃設計成一堂不需硬體、卻極有啟發性的課。這是本書認為最適合高中課堂的活動之一，大學部也能延伸為程式實作。

為什麼從掃地路線切入

覆蓋路徑規劃是一個「人人都有直覺、但一動手就發現不簡單」的問題。每個人都掃過地、拖過地，直覺上知道「要一排一排來、不要亂跑」——但要精確定義「怎麼排最有效率」、「遇到障礙怎麼繞」、「如何確保不遺漏」，就進入了演算法的領域。

紙上活動設計（高中適用）

34. 在方格紙上畫一個「房間」：外框是牆，內部放幾個障礙物（代表沙發、桌子），留一個門口。
35. 讓同學用筆規劃一條路線，要求「經過每一格至少一次、從門口出發回到門口」，並記錄總步數。
36. 比較全班的路線：誰的步數最少？最短路線與最長路線差多少？那些「多繞的步數」就是重複率的具體化。
37. 引入弓字形策略：示範如何把房間切成數個長條區域，每區走弓字形再連接，比較它與直覺路線的效率。

38. 加入變數：如果房間中央多一根柱子，弓字形路線要怎麼調整？如果門口在角落而非中間呢？

觀察家中掃地機器人（延伸作業）

39. 觀察並記錄自家（或影片中）掃地機器人的清掃路線：它是隨機式還是弓字形？它如何處理家具？
40. 拍下 App 上的地圖，標示出：哪些區域被分割成一個房間？有沒有明顯的地圖錯誤（第 13.7 節的病徵）？
41. 刻意製造挑戰：在地上放一雙拖鞋或一條（安全的）繩子，觀察機器人是否偵測到、如何反應。
42. 把它抱到另一個房間放下，觀察重定位過程（第 3 章綁架問題、第 11 章案例 11-2）。

大學部程式實作方向

- 用 Python 實作弓字形覆蓋規劃：輸入一張含障礙的佔據格地圖，輸出覆蓋路徑，並計算覆蓋率與重複率。
- 實作房間分割：用形態學運算或距離變換找出門口瓶頸，把地圖切成區塊。
- 在 ROS 模擬中，把第 14 或 15 章建好的地圖接上自製的覆蓋規劃器，讓虛擬機器人執行完整清掃。

至此，第四篇「2D SLAM 與掃地機器人」全部完成。我們從第 13 章的佔據格地圖出發，經過第 14 章 Gmapping 的粒子濾波、第 15 章 Cartographer 的圖優化與閉環，最後在本章看到這些技術如何在家庭中落地。同學應該能感受到：SLAM 不是遙不可及的尖端科技，它的核心版本正在你家客廳裡默默運轉。

下一篇，我們把維度從平面提升到立體。第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲、ICP 與 NDT 配準、LOAM 系列演算法，以及機器狗與巡檢機器人如何在複雜的

三維空間中定位建圖。難度會提升，但同學已經備妥了所有必要的基礎。

16.8 家用清掃機器人的下一步：2024–2026 年的四個轉向

本章前七節描述的是一套已經成熟的技術組合：2D 光達建圖、房間分割、弓字形覆蓋。這套組合在 2017 年之後基本定型，而真正的變化發生在它的邊界上——當產業發現「把地掃乾淨」這件事的技術瓶頸已經不在 SLAM，而在 SLAM 之外時，創新的方向就整個轉了彎。本節整理四個轉向。

一、從「繞開障礙」到「把障礙收走」

最戲劇性的變化發生在 2025 年初的 CES：Roborock 發表了 Saros Z70，在掃地機器人上裝了一支可折疊的五軸機械手臂 OmniGrip，能把地上 300 公克以下的小物件——襪子、小毛巾、衛生紙、拖鞋——夾起來放到指定位置，再回頭清掃那塊地板。

從本書的角度看，這件事的意義不在手臂本身，而在它對地圖的要求整個升級了。要夾起一隻襪子，機器人不只要知道「那裡有障礙」（佔據格做得到），還要知道「那是一隻襪子、它的三維形狀是這樣、可以夾、該放到哪裡」。這已經遠遠超出第 13 章佔據格的表達能力，進入語意地圖與物件層級表示的領域。換句話說，機械手臂逼出來的真正創新，是地圖格式的升級。

二、從幾何地圖到語意地圖

承接上一點：近年的研究主流，是在幾何地圖之上疊一層「這是什麼」的資訊。開放詞彙三維場景圖（open-vocabulary 3D scene graph）這一路的工作，把視覺語言模型的辨識能力接到三維地圖上，讓機器人能回應「去沙發旁邊那張桌子」這種用自然語言描述、而非座標指定的任務。

對掃地機器人而言，這條線的價值非常實際。16.4 節的房間分割只能靠幾何瓶頸切塊，遇到開放式廚房就失效；若地圖本身帶有物件標籤，「有流理台與冰箱的區域就是廚房」便成為

可用的判準。禁區也一樣——與其讓使用者在 App 上手動畫框，不如讓機器人自己認出寵物碗並繞開。本書第三篇的機率框架處理「那裡有沒有東西」，語意地圖處理的是「那是什麼」，兩者是疊加而非取代的關係。

三、覆蓋路徑規劃的標準化：從自家演算法到開源基礎設施

第三個轉向發生在軟體生態。覆蓋路徑規劃長年是各廠商的獨門配方，但 ROS 2 生態近年出現了通用的覆蓋規劃伺服器（opennav_coverage，建構於 Fields2Cover 之上），把本章談的策略整理成可設定的選項：路徑順序可選 BOUSTROPHEDON（弓字形）、SNAKE、SPIRAL（螺旋）；掃描角度可以指定，也可以用暴力搜尋自動找出最佳角度，而「最佳」的評分標準正好就是本章的三個指標——總長度、覆蓋率，或行數。

這份對應關係值得同學細看：16.4 節講的評估指標，在這套開源工具裡直接成為可選的目標函數；而延伸挑戰要同學比較的「弓字形 vs 螺旋式」，在這裡是一個設定參數。這說明本章的內容並不是產品觀察的雜談，而是一個有標準化工具支撐的工程領域。它原本是為農機（田間作業）設計的，但覆蓋問題的數學結構與室內清掃完全相同。

四、掃地機器人成為研究與教學的標準底盤

最後一個轉向與同學最有關。如案例 16-3 所述，掃地機器人底盤正逐漸成為低成本研究平台的首選：底盤成熟、馬達與里程計介面現成、體積適合室內、價格遠低於研究級機器人，而且頂部平坦容易加裝感測器支架。對於經費有限的大學專題與高中社團，這是一條把本書十六章理論真正跑起來的務實路徑。

本書的建議是：不必一開始就追求昂貴的設備。先用一台可程式化的掃地機底盤，加上一顆低成本 2D 光達，就能完整重現第 13 至 16 章的所有內容——建圖、閉環、房間分割、覆蓋規劃。把省下來的預算與時間，放在演算法與場域實驗上，那才是專題真正的價值所在。

四個轉向的共同訊號

硬體面（機械手臂）：清掃機器人開始「動手」，而不只是「繞開」。

表示面（語意地圖）：地圖從「哪裡有東西」升級為「那是什麼」，補上純幾何方法的盲點。

軟體面（開源覆蓋規劃）：覆蓋路徑規劃從各家配方變成可設定的標準元件。

教學面（研究底盤）：量產消費硬體回流成為低成本的研究與教學平台。

共同訊號：SLAM 本身已不是瓶頸。本章前七節的技術仍然是地基，但競爭已經移到地基之上——地圖裡還能放進什麼資訊，以及機器人拿這些資訊能做什麼。

承先啟後

第四篇到此完成。回頭看這四章的安排：第 13 章把掃描變成地圖，第 14 章用粒子濾波解開雞生蛋問題，第 15 章用圖優化讓歷史可以被修正，而第 16 章把這三件事放進一台真實產品裡，看它在 85 公分的門口、3 公分高的電線與一面鏡子面前如何應對。圖 16-1 到圖 16-3 的三組實驗與剖面，說的都是同一件事：演算法的優雅，最終要由真實世界的幾何來驗收。

從第 17 章起，我們把維度從平面提升到立體。第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲、ICP 與 NDT 配準、LOAM 系列演算法與巡檢應用。本章圖 16-3 那條孤零零的掃描平面，正是下一篇要解決的問題——當掃描不再只有一個高度，機器人終於能看見完整的世界。

公式或流程

一、覆蓋率與重複率

評估清掃效率的兩個核心指標：

$$覆蓋率 = 實際清掃到的面積 \ / \ 可清掃總面積 \times 100\%$$

$$重複率 = 實際行走距離 \ / \ 理論最短覆蓋距離$$

理論最短覆蓋距離約等於「可清掃面積 ÷ 刷寬」——這是完美弓字形不重複走完全區的距離。重複率為 1 代表完美無重複，實務上因轉彎、繞障礙與返回充電，通常落在 1.2 至 2.0 之間；隨機碰撞式則可能高達 5 以上。

二、弓字形路徑的行數

在一個寬 W、長 L 的無障礙矩形區域中，以刷寬 w 走弓字形所需的行數與距離為：

$$行數\ n = \lceil W / w \rceil$$

$$總距離 \approx n \times L + (n - 1) \times w$$

第二項是每次轉行時橫向移動的距離。這條式子說明了為何刷寬越大效率越高——行數減少，轉彎次數也減少。這也是產品設計上「加寬刷頭」的動機。

三、掃地機器人的完整運作流程

圖 16-4 掃地機器人一次清掃的完整流程

① 從充電座出發，載入既有地圖（首次則進入 SLAM 建圖模式）

↓

② 定位：以 AMCL 類方法確認自己在地圖中的位置（第 11 章）

↓

③ 房間分割與任務規劃：決定清掃順序，排除禁區（16.4 節）

↓

④ 覆蓋路徑規劃：在當前房間內產生弓字形路線

↓

⑤ 執行清掃：沿路徑移動，同時

- 持續定位與地圖更新（SLAM）
- 即時避障（相機 / 3D ToF / 碰撞 / 懸崖感測器）
- 記錄已清掃區域

↓

⑥ 電量不足？→ 規劃返回充電座，充飽後回到中斷點續掃

↓

⑦ 全部房間完成 → 返回充電座，上傳地圖與清掃紀錄

四、房間分割的基本思路

圖 16-5 房間分割的距離變換法

① 對佔據格地圖執行距離變換：每個自由格記錄它到最近障礙的距離

↓

② 房間中央距離值大（空曠）、門口與走道距離值小（狹窄）

↓

③ 取高於門檻的區域作為各房間的「核心種子」

↓

④ 由各核心向外擴張，直到相遇為止 → 邊界即為房間分界

↓

⑤ 輸出分割後的房間區塊，供使用者命名與個別指定清掃

案例

案例 16-1 二十年演進：從 Roomba 到 Physical AI

掃地機器人的二十年，是一部機器人技術大眾化的縮影。2002 年第一代 Roomba 以隨機碰撞策略問世——沒有地圖、沒有定位，純粹是第 3 章「行為式架構」的商業化。2010 年 Neato 首度導入 2D 光達與 SLAM。2017 年小米以親民價格普及光達機型（第 1 章案例 1-1）。

而到了 2026 年，情況又有了質變。這一代的家用機器人被歸類為「實體 AI（Physical AI）」——它們不再只是靠隨機碰撞轉向的旋轉刷具，而是執行裝置端 AI，能建立地圖、避開障礙，並適應真實居家生活的樣態。它們用光達與相機 SLAM 掃描房間、建立牆面家具與常用動線的 2D 或 3D 地圖；地圖建好後能為房間貼標籤、排除禁區，甚至記住哪些區域需要更頻繁清掃。

這條演進線清楚顯示：SLAM 是基礎，但真實產品會不斷疊加感測器與 AI 來突破單一技術的限制。從第 13 章的佔據格，到本章的語意標籤與行為適應，同學看到的正是本書從第四篇通往第九篇（AI 語意導航）的完整脈絡。

案例 16-2　為什麼你家的掃地機器人總是撞到那張椅子

這是一個幾乎每位使用者都遇過、也最能印證本書理論的日常現象。機器人能靈巧繞開沙發與牆壁，卻總是一頭撞上餐桌的椅腳。

原因是三重的。其一是第 8 章的角解析度問題：細瘦的椅腳在數公尺外可能整個落在兩束雷射之間被漏測。其二是掃描平面高度：有些椅腳上寬下窄，光達掃到的高度剛好是最細的部分。其三是動態性：椅子每天被拉到不同位置，地圖上記錄的舊位置早已失效。

對策也對應三個層次：近距離重掃（角解析度隨距離改善）、加裝 3D ToF 或相機看到完整形狀、以及讓地圖保持可塑性以反映椅子的移動（第 13.5 節）。下次看到機器人撞椅腳，不妨想想這三個原因分別對應本書的哪一章——這是最貼身的 SLAM 複習。

案例 16-3　掃地機器人平台在研究與教學中的角色

有趣的是，掃地機器人不只是消費產品，也是研究與教學的實用平台。有研究團隊以可程式化的掃地機器人底盤作為無人地面載具，搭配低成本 360 度 2D 光達，在 ROS 框架下實作 SLAM 與覆蓋路徑規劃研究。

這類平台的優勢很實際：底盤成熟穩定、有現成的馬達與里程計介面、體積小適合室內、價格遠低於研究級機器人，而且結構上容易加裝額外的感測器支架。對大學的專題或高中的進階社團而言，這是一條可行的低成本路徑——用消費級底盤加上教學型光達與單板電腦，就能組出一台能跑第 14、15 章演算法的研究平台。

這個案例呼應第 8 章 8.7 節的教學平台討論，也給同學一個啟示：做機器人研究不一定要昂貴的設備，善用成熟的消費級硬體作為基礎，把心力放在演算法與應用創新上，往往是

更聰明的策略。

實作活動

實作 16-1　覆蓋路徑規劃：從紙上到程式

目的：透過紙上推演、實地觀察與程式實作三個層次，理解覆蓋路徑規劃的策略與評估指標，並把第四篇的所有理論連結到生活經驗。

第一部分：紙上覆蓋規劃（全體，不需器材）

40. 在方格紙上畫一個 12×10 格的房間，內部放兩個 2×3 格的障礙物（沙發、桌子），在一側邊緣留一格作為充電座位置。
41. 每人獨立規劃一條路線，要求經過每一個自由格至少一次，並回到充電座。記錄總步數。
42. 全班統計：最少步數是多少？最多是多少？計算各自的重複率（總步數 ÷ 自由格數）。
43. 示範弓字形策略：把房間依障礙切成數個長條區，每區走弓字形再連接。比較它與直覺路線的步數差異。
44. 挑戰題：如果把兩個障礙物移到房間正中央並排，弓字形要如何調整？畫出你的解法。

第二部分：觀察真實掃地機器人（家庭作業）

45. 觀察家中（或網路影片中）掃地機器人的清掃路線，判斷它是隨機式還是規劃式，並描述它處理家具的方式。
46. 截圖 App 上的房間地圖，標示：房間分割是否合理？有無第 13.7 節的地圖病徵（變形、斷裂、漂移）？
47. 低矮障礙測試：在地上放一雙拖鞋（安全物品），觀察機器人是繞開、推開還是卡住，並用 16.6 節的知識解釋。

48. 綁架測試：清掃中途把機器人移到另一個房間，計時它花多久重新定位成功（第 3 章綁架問題、第 11 章案例 11-2）。

第三部分：程式實作（大學部）

49. 用 Python 讀入一張佔據格地圖（可用第 13 章實作產生的，或自行定義），實作弓字形覆蓋路徑規劃：把地圖切成無障礙長條區，在每區產生來回路徑，再連接各區。

50. 計算並輸出覆蓋率與重複率，用 matplotlib 把路徑畫在地圖上視覺化。

51. 實作房間分割：依圖 16-5 的距離變換法，把地圖自動切分成房間，以不同顏色顯示。

52. 加入禁區功能：讓使用者指定一個矩形區域，規劃時自動排除，驗證路徑確實繞開。

53. （進階）在 ROS 模擬中，把第 14 或 15 章建好的地圖接上你的覆蓋規劃器，讓虛擬機器人實際執行完整清掃。

討論問題

13. 第一部分中，弓字形路線比直覺路線省了多少步數？這個差距在真實的一百坪住宅中，會轉換成多少清掃時間？

14. 觀察真實機器人時，你發現了哪些地圖病徵？依第 13.7 節判斷，它們最可能的成因是什麼？

15. 綁架測試花了多久重新定位？若機器人被放在一個與原位置外觀相似的房間（例如兩間格局相同的臥室），你預期會發生什麼？（提示：第 11 章的多峰分布）

16. 依表 16-3，如果要為「養寵物的家庭」設計一台掃地機器人，你會優先加強哪些感測器？為什麼？

延伸挑戰（大學部）

其一：比較兩種覆蓋策略——弓字形與螺旋式（從外往內繞）——在同一張地圖上的覆蓋率、重複率與轉彎次數，分析各自適合什麼形狀的房間。其二：實作「斷點續掃」功能—

—在路徑執行到一半時中斷，記錄已完成的區域，重新啟動後只清掃剩餘部分，並驗證覆蓋率仍達 100%。這個功能需要同時用到定位、地圖與任務狀態管理，是整合第四篇所有知識的好題目。

延伸閱讀

22. H. Choset, "Coverage for Robotics – A Survey of Recent Results," Annals of Mathematics and Artificial Intelligence, 2001.（覆蓋路徑規劃的經典綜述，含 Boustrophedon 分解法）
23. E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013.（覆蓋規劃的完整回顧，適合專題參考）
24. R. Bormann et al., "Room Segmentation: Survey, Implementation, and Analysis," ICRA, 2016.（房間分割方法的比較研究，含開源實作）
25. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（佔據格與蒙地卡羅定位的理論基礎，呼應本章的建圖與重定位）
26. R. Chuchro, "Simulating Coverage Path Planning with Roomba," arXiv, 2022.（以強化學習探索覆蓋規劃，並與掃地機器人內建演算法比較）
27. ROS Navigation2 官方文件之 Coverage Server 說明（docs.nav2.org）.（ROS 2 生態中的覆蓋路徑規劃工具）
28. OpenCV 官方文件之 Distance Transform 與形態學運算教學.（房間分割實作的技術基礎）

小結

22. 掃地機器人需要 SLAM 才能回答「哪裡掃過了、何時算結束、怎麼回家」；SLAM 帶來系統性清掃、斷點續掃、使用者互動與多次清掃一致性四項能力，是本書前十五章的實體總複習。

23. 隨機碰撞式是反應式架構的商業化（2002 年 Roomba），路線雜亂、覆蓋不保證；SLAM 規劃式建圖後走弓字形路徑，覆蓋率高、重複率低，但地圖錯誤會直接導致清掃錯誤。

24. 光達掃地機以低成本 2D 光達建佔據格地圖，首次清掃執行 SLAM、後續清掃載入地圖定位；現代高階機型光達與 AI 相機並用——光達建圖定位、相機辨識地板物體，正是「光達給幾何、相機給語意」的商業實踐。

25. 房間分割靠找出門口等空間瓶頸把地圖切塊，已觸及語意地圖概念；禁區是在代價地圖上注入使用者知識；覆蓋路徑規劃求「走遍每一格」而非「抵達某點」，以覆蓋率、重複率、清掃時間三指標評估。

26. 感測器組合包含 2D 光達、IMU、輪速編碼器、碰撞、懸崖、紅外 / 超音波、AI 相機與 3D ToF；如此多元的原因是 2D 光達的高度盲區——低矮障礙、樓梯落差、玻璃鏡面都需要其他感測器補足。

27. 家庭五大挑戰：細瘦桌椅腳（角解析度不足）、鏡面與落地窗（雷射穿透反射）、低矮動態障礙（在掃描平面之下）、環境每天在變（地圖需兼顧記憶與遺忘）、地面材質變化（里程計品質下降）。

28. 覆蓋路徑規劃是「人人有直覺、一動手就不簡單」的好教材，可從紙上規劃、觀察真實機器人到程式實作分三層次教學，是連結理論與生活的最佳橋樑。

第四篇「2D SLAM 與掃地機器人」到此完成。我們從佔據格地圖出發，經過 Gmapping 的粒子濾波與 Cartographer 的圖優化閉環，最後看到這些技術如何在家庭中落地——SLAM 不是遙不可及的尖端科技，它正在你家客廳裡默默運轉。下一篇，我們把維度從平面提升到立體：第五篇「3D 光達 SLAM 與自主巡檢」將介紹三維點雲處理、ICP 與

NDT 配準、LOAM 系列演算法，以及機器狗與巡檢機器人如何在複雜的三維空間中定位建圖。